

OS

Operating System

Hardware and software:

Hardware: The physical components of a computer are called hardware. A physical component be electronic, electrical, mechanical or optical etc. Ex: hard disks, microprocessors mouse, keyboard, RAM, ROM etc.

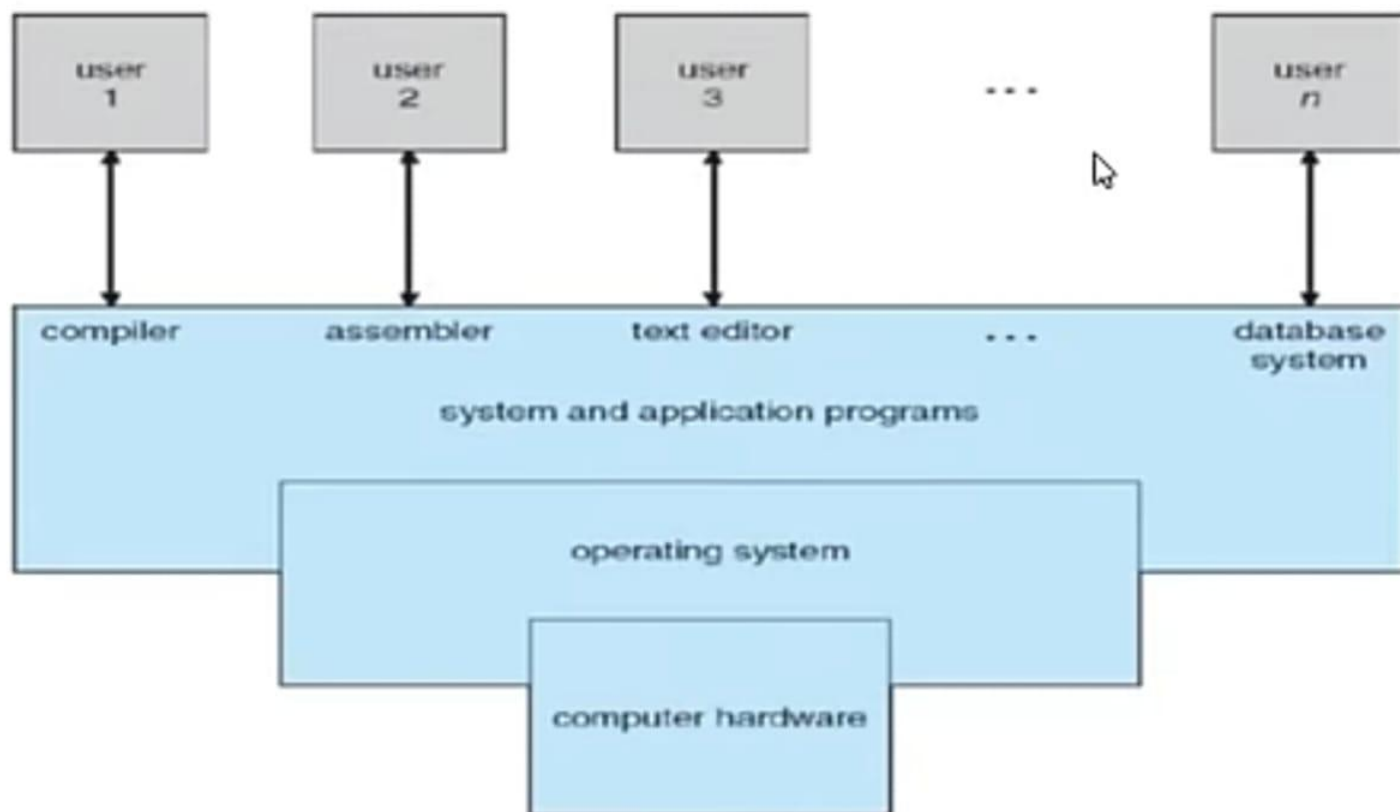
Software: A sequence of instructions given to a computer to perform a particular task is called a program. There are two types of softwares:

- 1. Application Software:** Application software is a set of one or more programs designed to carry out operations for a specific applications like games, library management system, MS-word, Notepad etc.
- 2. System Software:** System softwares are those which provide the environment for the execution of the application softwares like Operating system, compiler, device drivers etc.

Operating System:

An **operating system** is a program that manages the computer hardware. It also provides a basis for application programs and acts as an intermediary between the computer user and the computer hardware.

- The **operating system** controls and coordinates the use of the hardware among the various application programs for the various users.



- The **hardware**—the **central processing unit (CPU)**, the **memory**, and the **input/output (I/O) devices**—provides the basic computing resources for the system.
- The **application program** define the ways in which these resources are used to solve users' computing problems.

Kernel

- Kernel is central component of an operating system that manages operations of computer and hardware.
- It basically manages operations of memory and CPU time.
- Kernel acts as a bridge between applications and data processing performed at hardware level using inter-process communication and system calls.

Operating System Components:

Many modern operating systems have the following components.

- Process Management
- Main Memory Management
- File Management
- I/O System Management
- Secondary Management
- Networking
- Protection System
- Command-Interpreter System

A. Process Management

- A process is a program in execution. A **program itself is not a process** rather it is a passive entity.
- A process needs certain resources, including **CPU time, memory, files, and I/O devices**, to accomplish its task. These resources are either given to the **process when it is created or when it is running**. When the process completes, the OS reclaims all the resources.
- The operating system is responsible for the following activities in connection with process management.
 - Process creation and deletion.
 - Process suspension and resumption.
 - Provision of mechanisms for:

B. Main Memory Management

- **Memory is a large array of words or bytes, each with its own address. It is a repository of quickly accessible data shared by the CPU and I/O devices.**
- **The operating system is responsible for the following activities in connections with memory management:**
 - **Keep track of which parts of memory are currently being used and by whom.**
 - **Decide which processes to load when memory space becomes available.**
 - **Allocate and deallocate memory space as needed.**

C. File Management:

- Most visible component of OS. Computers can store information on several different types of **physical media** (e.g. magnetic tap, Hard disk etc).
- **A file a logical storage unit.** A file is a collection of related information defined by its creator. Commonly, **files represent programs (both source and object forms) and data.**
- The operating system is responsible for the following activities in connections with file management:
 - File creation and deletion.
 - Directory creation and deletion.
 - Support of primitives for manipulating files and directories.
 - Mapping files onto secondary storage.

D. I/O System Management

- The I/O system consists of:
 - A buffer-caching system
 - A general device-driver interface
 - Drivers for specific hardware devices

E. Secondary Storage management

- Since main memory (*primary storage*) is volatile and too small to accommodate all data and programs permanently, the computer system must provide ***secondary storage to back up main memory.***
- Most modern computer systems use **disks** as the principle on-line storage medium, for both programs and data.
- The operating system is responsible for the following activities in connection with disk management:
 - Free space management
 - Storage allocation
 - Disk scheduling

F. Networking (Distributed Systems)

- Distributed System is a collection of **autonomous computer systems** that are physically separated but are connected by a **centralized computer network** that is equipped with distributed system software.
- The **processors** in the system are connected through a **communication network**.
- **Communication** takes place using a *protocol*.
- A distributed system provides **user access** to **various system resources**.
- Access to a shared resource allows:
 - Computation speed-up

→ Enhanced reliability = |

G. Protection System

- *Protection* refers to a mechanism for **controlling access** by programs, processes, or users to both system and user resources.
- The protection mechanism must:
 - Distinguish between **authorized and unauthorized usage**.
 - Specify the **controls to be imposed**.
 - Provide a means of **enforcement**.

H. Command-Interpreter System

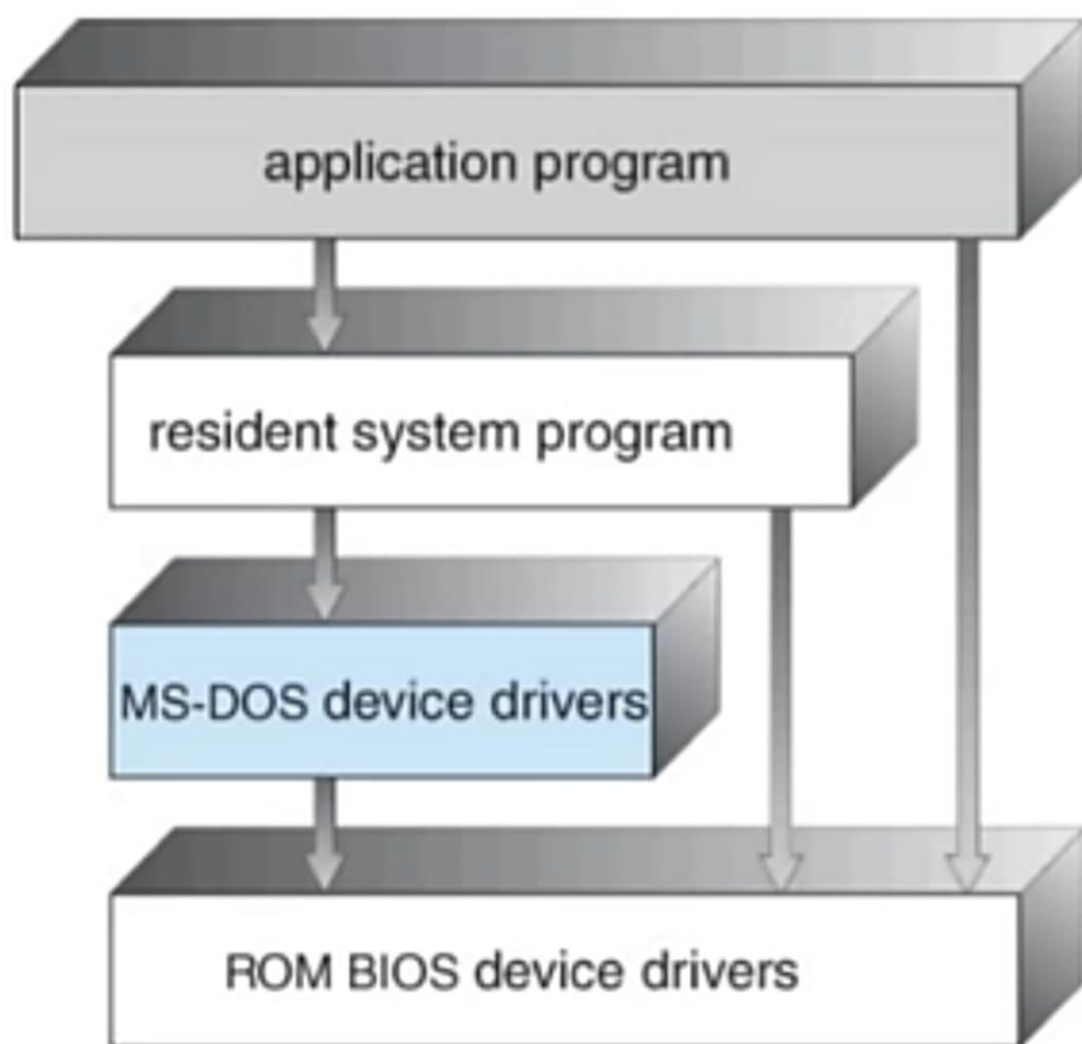
- The command interpreter is the primary interface between the user and the rest of the system.
- Command Interpreter System executes a user command by calling one or more number of underlying system programs or system calls.
 - Process creation and management
 - I/O handling
 - Secondary-storage management
 - Main-memory management
 - File-system access
 - Protection
 - Networking
- The program that reads and interprets control statements:

Operating System Structure:

1. Simple Structure
2. Monolithic Structure
3. Layered Approach Structure
4. Micro-Kernel Structure
5. Modules Structure

1. Simple Structure:

- do not have well defined structure
- These are small, simple and limited systems.



1. Simple Structure:

- do not have well defined structure
- These are small, simple and limited systems.
- Example-MS-DOS operating system has four layers - ROM BIOS device drivers, MS-DOS device drivers, application programs, and system programs.
- **Advantages-**
 - Each level can be defined independently and, when necessary, can interact with one another.
 - It will be simpler to design, manage, and update. Because of this, simple structures can be used to build constrained systems that are less complex.

Disadvantages-

- When a user program fails, the operating system as whole crashes.
- Layers are interconnected, and in communication with one another, there is no abstraction or data hiding.
- The operating system's operations are accessible to layers, which can result in data tampering and system failure.

2. Monolithic Structure:

- The Monolithic operating System in which the kernel acts as a manager by managing all things like file management, memory management, device management, and operational processes of the Operating System.
- The kernel is the heart of a computer operating system (OS). Kernel delivers basic services to all other elements of the System. It serves as the primary interface between the Operating System and the hardware.

(the users)

shells and commands
compilers and interpreters
system libraries

system-call interface to the kernel

signals terminal
handling
character I/O system
terminal drivers

file system
swapping block I/O
system
disk and tape drivers

CPU scheduling
page replacement
demand paging
virtual memory

kernel interface to the hardware

terminal controllers
terminals

device controllers
disks and tapes

memory controllers
physical memory

- **Advantages –**

- Layering is unnecessary and the kernel alone is responsible for managing all operations, it is easy to design and execute.

- **Disadvantages-**

- The monolithic kernel's services are interconnected in one address space and have an impact on one another, so if any of them malfunctions, the entire system does as well.
 - It is not adaptable. Therefore, launching a new service is difficult.
-

3. Layered Structure

- The operating system is divided into a number of layers (levels), each built on top of lower layers. The bottom layer (layer 0), is the hardware; the highest (layer N) is the user interface.
- The operations (routines) can be invoked by higher-level layers. The layer itself can invoke operations on lower-level layers.

Layered structure:

Layered Structure of the THE OS

- A layered design was first used in THE operating system.

Its six layers are as follows:

layer 5:	user programs
layer 4:	buffering for input and output
layer 3:	Process management
layer 2:	memory management
layer 1:	CPU scheduling
layer 0:	hardware

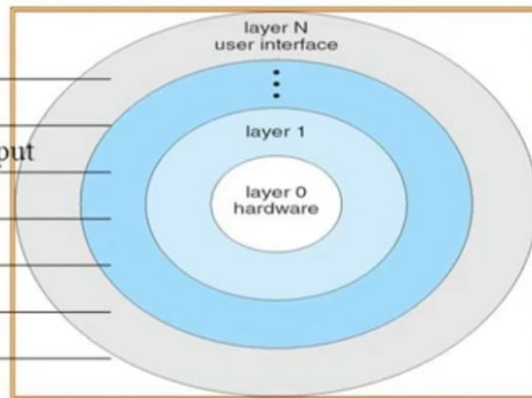
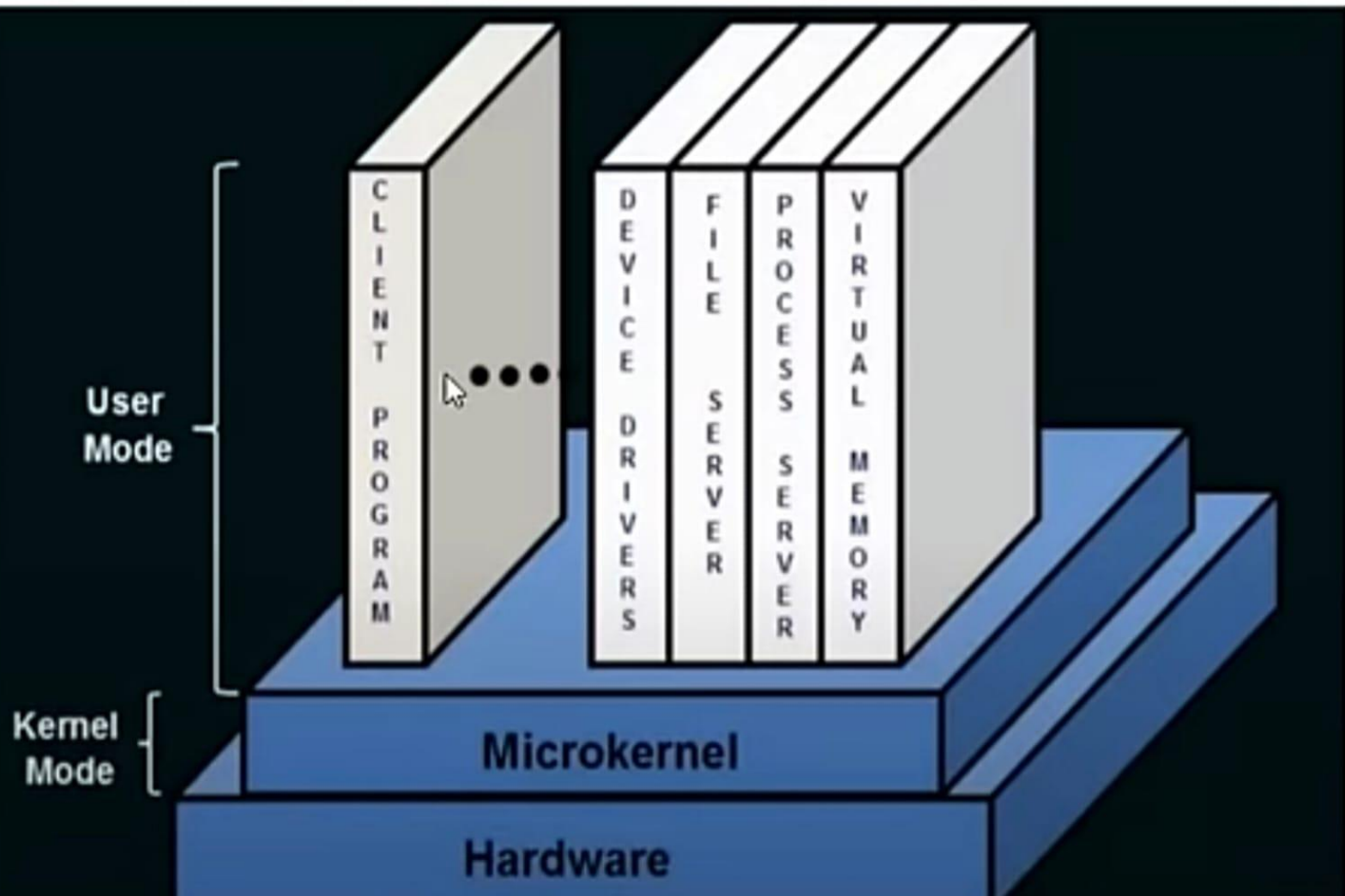


Fig. 4 A layered Operating System

- Advantages-
 - Layered approach provides modularity. With modularity, layers are selected such that each layer uses functions (operations) and services of only lower-level layers.
 - Debugging is easier as lower layers are debugged, and then upper layers are checked.
- Disadvantages-
 - It takes careful planning to construct the layers since higher layers only utilize the functions of lower

3. Micro kernels Structure:

- Kernel using microkernel approach (smaller kernels) - Remove all non-essential components from the kernel & implementing them as system & user level programs.
- Advantages-
 - Each Micro-Kernel is made independently and is isolated from other Micro-Kernels. So this makes the system more secure and reliable.
 - If any Micro-Kernel fails, then the remaining operating System remains untouched and works fine.



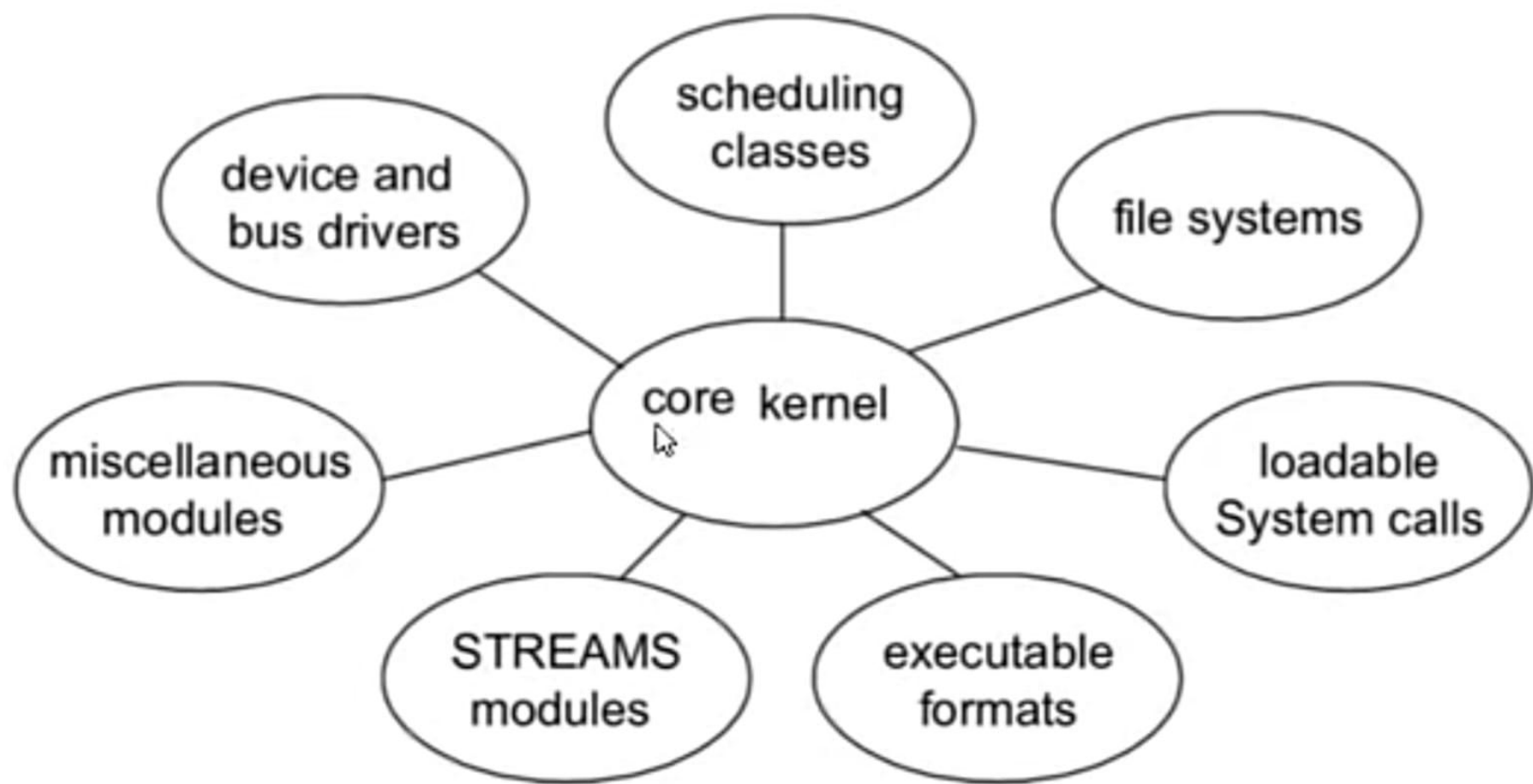
- Disadvantages:
 - Increased inter-module communication reduces system performance/ system overhead..
 - System is complex to be constructed.

Example: Windows NT

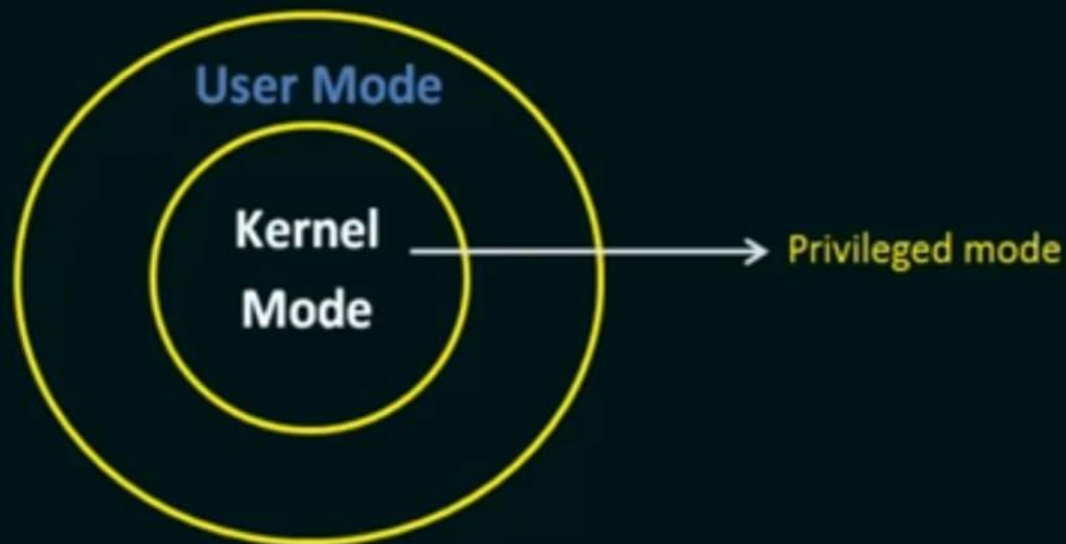
4. Modular Structure-

The kernel has only set of core components and other services are added as dynamically loadable modules to the kernel either during run time or boot time.

A layer can communicate other with the help of kernel.



System calls provide an interface to the services made available by an Operating System.



- System call is the programmatic way in which a computer program requests a service from the kernel of the operating system.
- These calls are generally available as routines written in C and C++.

Example of a System Call sequence for writing a simple program to read data from one file and copy them to another file:



Write prompt to screen

Accept Input

Open Input File

If File doesn't exist, ABORT

Create Output File

If File exists, ABORT

Loop

Read from Input File

until Read fails

Write to Output File

Close Output File

Write completion message to screen

Terminate normally

Types of System Calls

System calls can be grouped roughly into five major categories:

1. Process Control
 2. File Manipulation
 3. Device Management
 4. Information Maintenance
 5. Communications
- 

1. Process Control



- end, abort
- load, execute
- create process, terminate process
- get process attributes, set process attributes
- wait for time
- wait event, signal event
- allocate and free memory

2. File Manipulation

- create file, delete file
- open, close
- read, write, reposition
- get file attributes, set file attributes

3. Device Manipulation

- request device, release device
- read, write, reposition
- get device attributes, set device attributes
- logically attach or detach devices

4. Information Maintenance

- get time or date, set time or date
- get system data, set system data
- get process, file, or device attributes
- set process, file, or device attributes

5. Communications



- create, delete communication connection
- send, receive messages
- transfer status information
- attach or detach remote devices

Threads

A thread is a basic unit of CPU utilization.

It comprises

A thread ID

A program counter

A register set and

A stack

It shares with other threads belonging to the same process its code section, data section, and other operating-system resources, such as open files and signals.

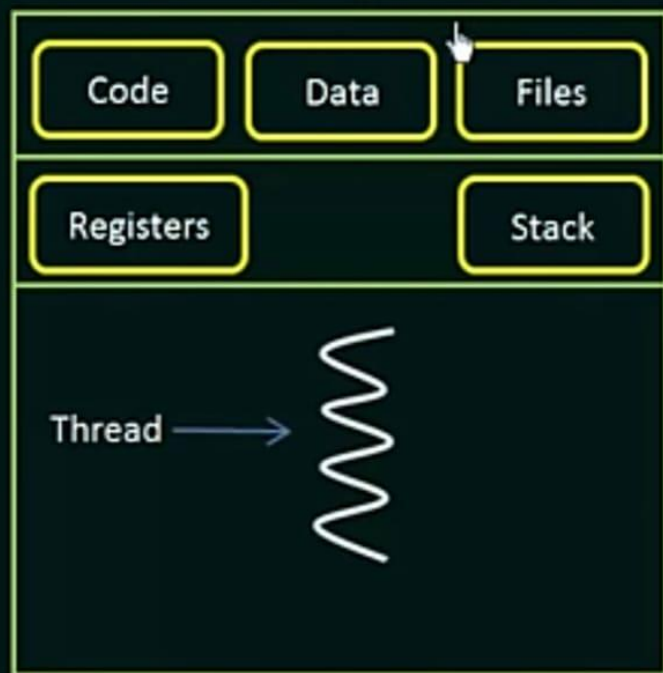
A traditional / heavyweight process has a **single thread** of control. 

If a process has **multiple threads** of control, it can perform **more than one task at a time.** 

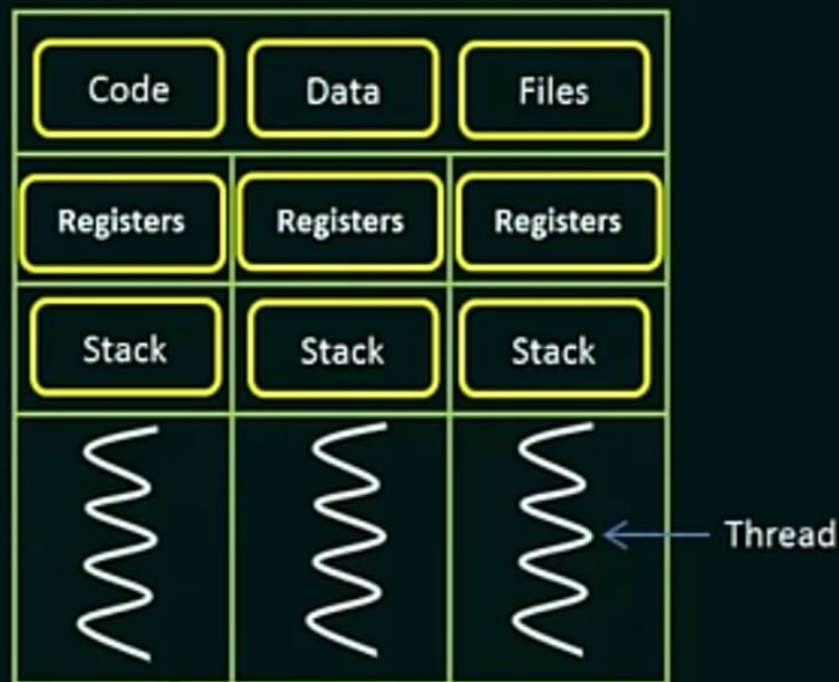
It shares with other threads belonging to the same process its code section, data section, and other operating-system resources, such as open files and signals.

A traditional / heavyweight process has a **single thread** of control.

If a process has **multiple threads** of control, it can perform **more than one task at a time**.



Single-threaded process



Multi-threaded process

The benefits of multithreaded programming can be broken down into four major categories:

Responsiveness

Multithreading an interactive application may allow a program to continue running even if part of it is blocked or is performing a lengthy operation, thereby increasing responsiveness to the user.

Resource sharing

By default, threads share the memory and the resources of the process to which they belong. The benefit of sharing code and data is that it allows an application to have several different threads of activity within the same address space.

Economy

Allocating memory and resources for process creation is costly. Because threads share resources of the process to which they belong, it is more economical to create and context-switch threads.

Utilization of multiprocessor architectures

The benefits of multithreading can be greatly increased in a multiprocessor architecture, where threads may be running in parallel on different processors. A single-threaded process can only run on one CPU, no matter how many are available. Multithreading on a multi-CPU machine increases concurrency.

Multithreading Models and Hyperthreading

Types of Threads:

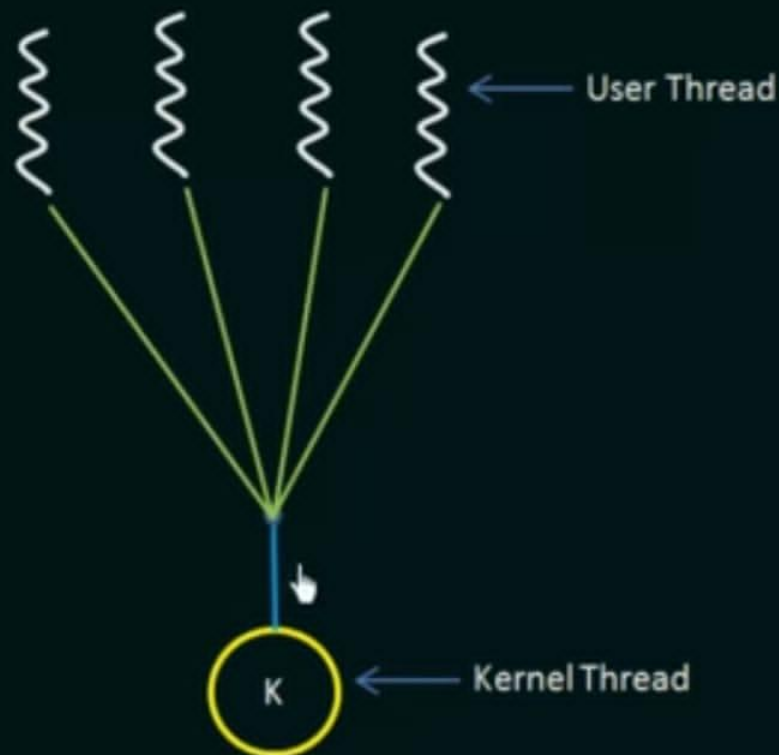
- 1) **User Threads** - Supported above the kernel and are managed without kernel support.
- 2) **Kernel Threads** - Supported and managed directly by the operating system.

Ultimately, there must exist a relationship between user threads and kernel threads.

There are three common ways of establishing this relationship:

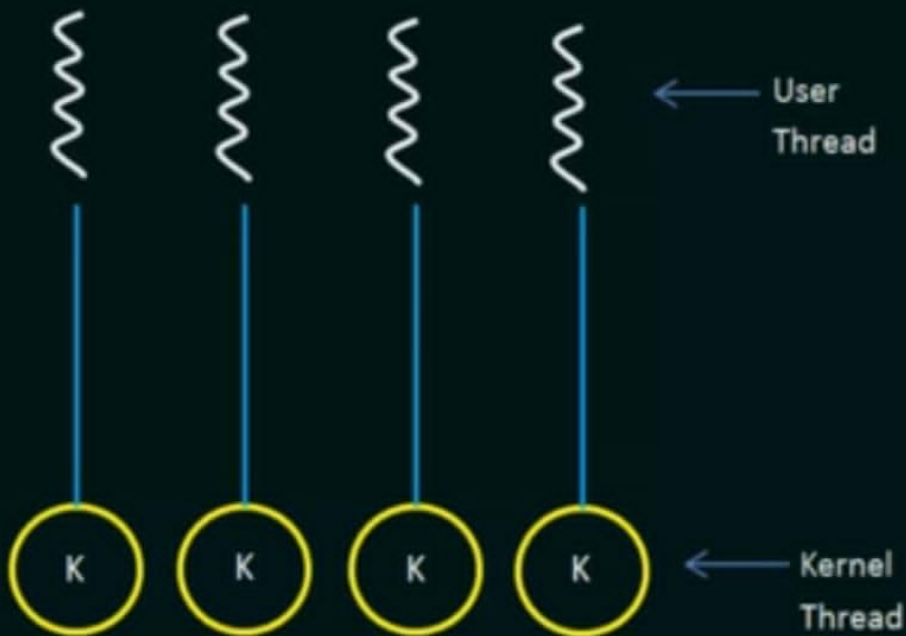
1. Many-to-One Model
2. One-to-One Model
3. Many-to-Many Model

Many-to-One Model



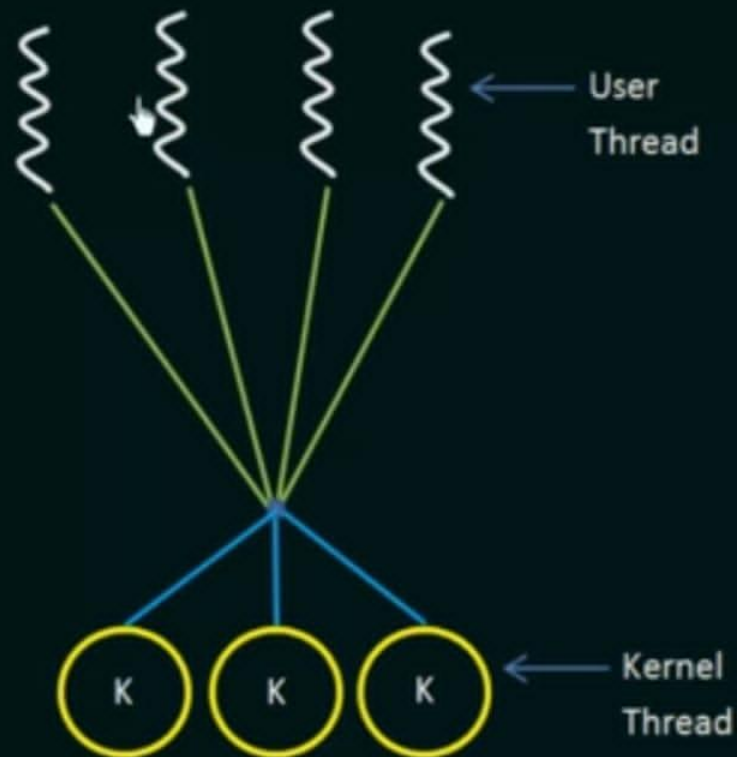
- Maps many user-level threads to one kernel thread.
- Thread management is done by the thread library in user space, so it is efficient.
- The entire process will block if a thread makes a blocking system call.
- Because only one thread can access the kernel at a time, multiple threads are unable to run in parallel on multiprocessors.

One-to-One Model



- Maps each user thread to a kernel thread.
- Provides more concurrency than the many-to-one model by allowing another thread to run when a thread makes a blocking system call;
- Also allows multiple threads to run in parallel on multiprocessors.
- Creating a user thread requires creating the corresponding kernel thread.
- Because the overhead of creating kernel threads can burden the performance of an application, most implementations of this model restrict the number of threads supported by the system.

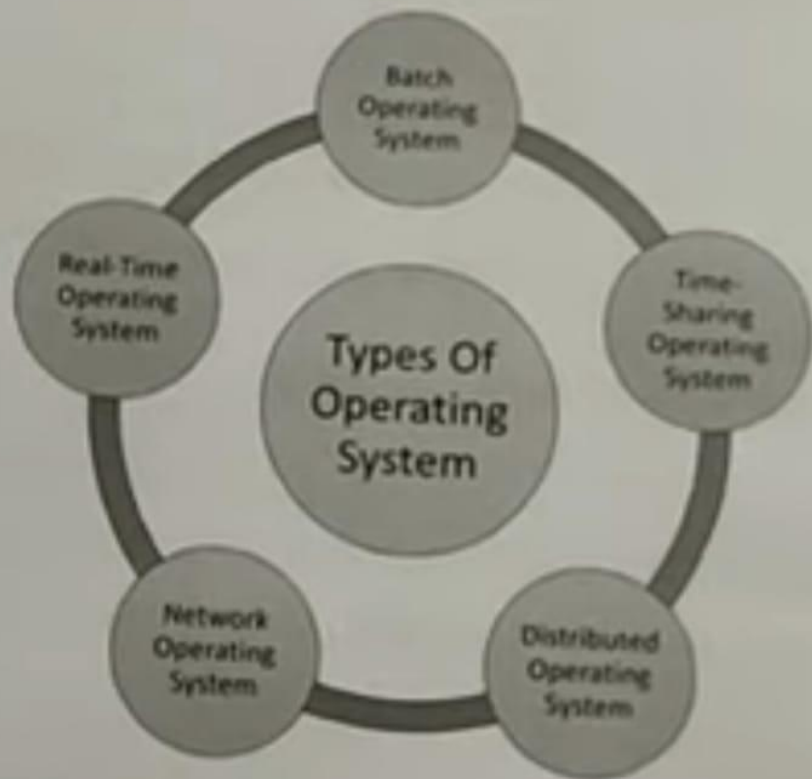
Many-to-Many Model



- Multiplexes many user-level threads to a smaller or equal number of kernel threads.
- The number of kernel threads may be specific to either a particular application or a particular machine.
- Developers can create as many user threads as necessary, and the corresponding kernel threads can run in parallel on a multiprocessor.
- Also, when a thread performs a blocking system call, the kernel can schedule another thread for execution.

TYPES OF OPERATING SYSTEM

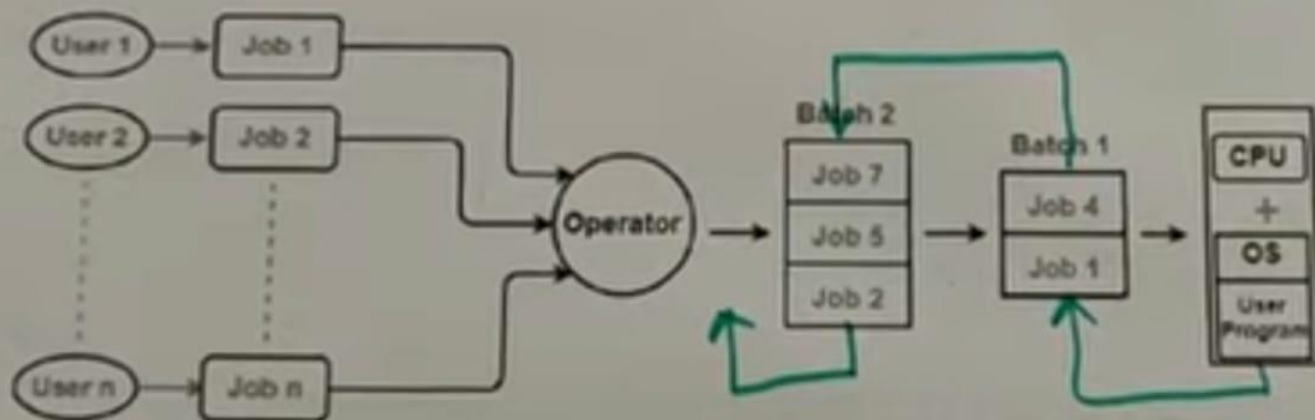
- Batch Operating System
- Time-Sharing Operating System
- Distributed Operating System
- Network Operating System
- Real-Time Operating System



TYPES OF OPERATING SYSTEM

➤ Batch Operating System:-

- ❖ In a Batch Operating System, the similar jobs are grouped together into batches with the help of some operator and these batches are executed one by one.

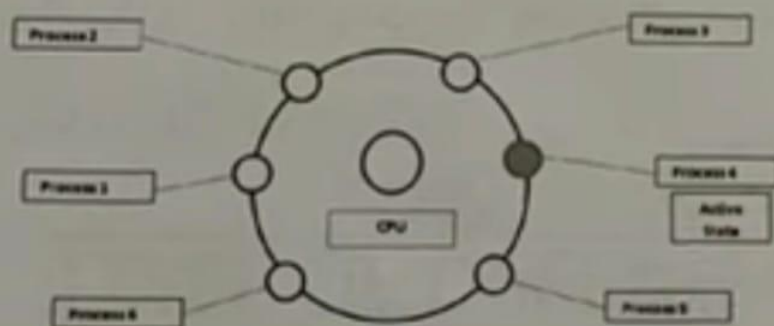


Batch Operating System

TYPES OF OPERATING SYSTEM

➤ Time-Sharing Operating System:-

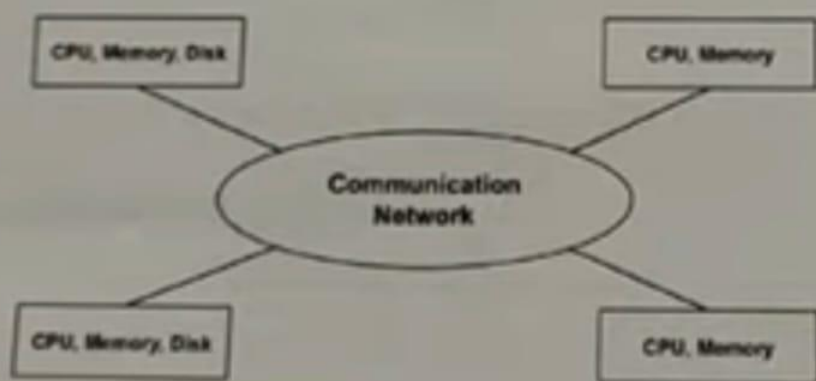
- ❖ In Time-Sharing Operating System Each task is given some time to execute so that all the tasks work smoothly.
- ❖ Each user gets the time of CPU as they use a single system.
- ❖ These systems are also known as Multitasking Systems. The task can be from a single user or different users also.



TYPES OF OPERATING SYSTEM

➤ **Distributed Operating System:-**

- ❖ In a Distributed Operating System, we have various systems and all these systems have their own CPU, main memory, secondary memory, and resources.
- ❖ These systems are connected to each other using a shared communication network. Here, each system can perform its task individually.



TYPES OF OPERATING SYSTEM

➤ **Network Operating System:-**

- ❖ These systems run on a server and provide the capability to manage data, users, groups, security, applications, and other networking functions.
- ❖ These types of operating systems allow shared access of files, printers, security, applications, and other networking functions over a small private network.



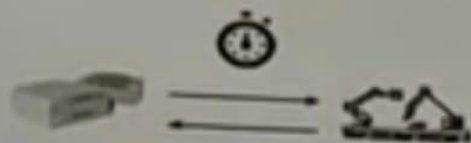
TYPES OF OPERATING SYSTEM

➤ Real-Time Operating System:-

- ❖ In Real Time Operating Systems, each job carries a certain deadline within which the Job is supposed to be completed, otherwise the huge loss will be there or even if the result is produced then it will be completely useless.

There are two types of Real-Time Operating System

- ❖ Hard Real-Time Operating Systems
- ❖ Soft Real-Time Operating Systems



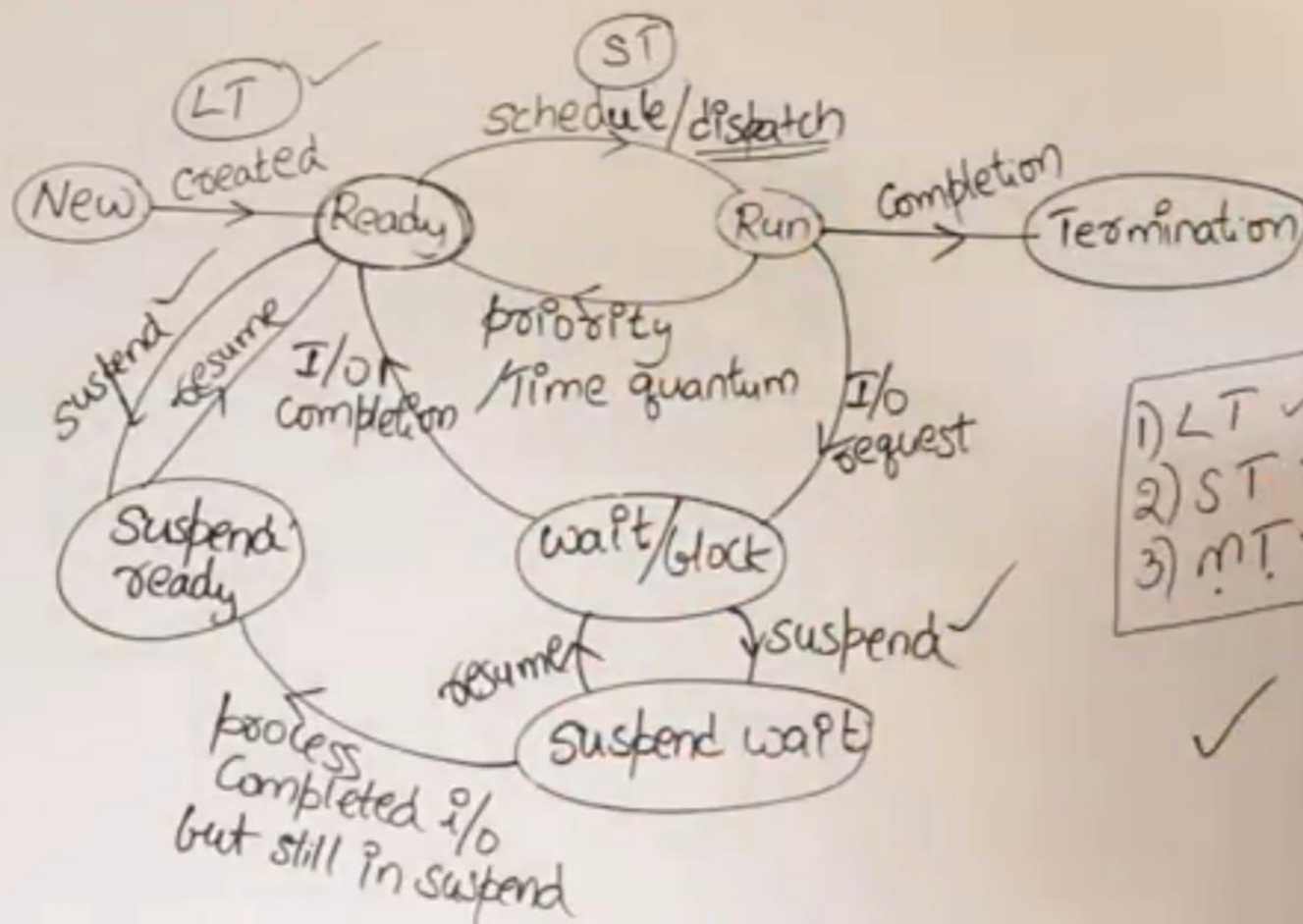
Real-Time Operating Systems

UNIT - II

Process Concept

Process:

- A **process** is an executing program, including the current values of the **program counter, registers, and variables**.
- Difference between a process and a program is that the program is a **group of instructions** whereas the **process is the activity**.
- Process can be described:
 - ❖ **I/O Bound Process**- spends more time doing I/O then computation.
 - ❖ **CPU Bound Process**- spends more time doing computation



New- The process is being created.

Running- Instructions are being executed.

Waiting- waiting for some event to occur (such as an I/O completion etc).

Ready- The process is waiting to be assigned to a processor.

Terminated- The process has finished execution.

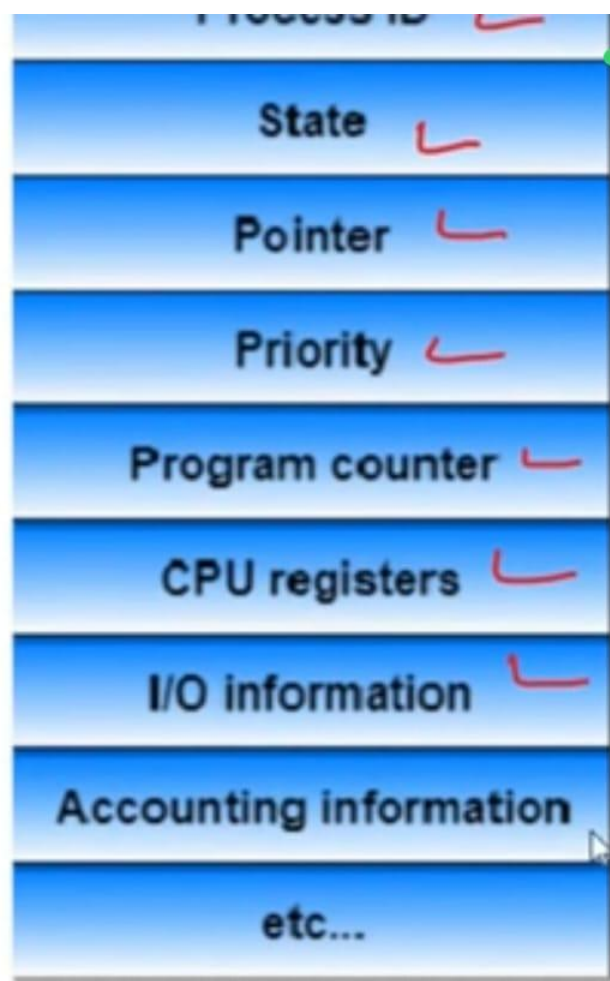
Process/Task Control Block (PCB):

In the OS, each process is represented by its PCB (Process Control Block). The PCB, generally contains the following information:

- **Process ID** Unique id for Process
- **Process State:** The state may be new, ready, running, waiting, halted, and so on.
- **Pointer:** points to parent process.
- **Priority:** Process Priority for CPU Scheduling.
- **Program Counter:** indicates the address of the next instruction to be executed.



- **Process ID** Unique id for Process
- **Process State:** The state may be new, ready, running, waiting, halted, and so on.
- **Pointer:** points to parent process.
- **Priority:** Process Priority for CPU Scheduling.
- **Program Counter:** indicates the address of the next instruction to be executed.
- **CPU Register:** String
- **I/O Status Info:** list of I/O devices allocated in process.
- **Accounting Info.:** amount of CPU used.
- **Memory Management Information:** page tables etc.



Operations on Process

Process Creation:

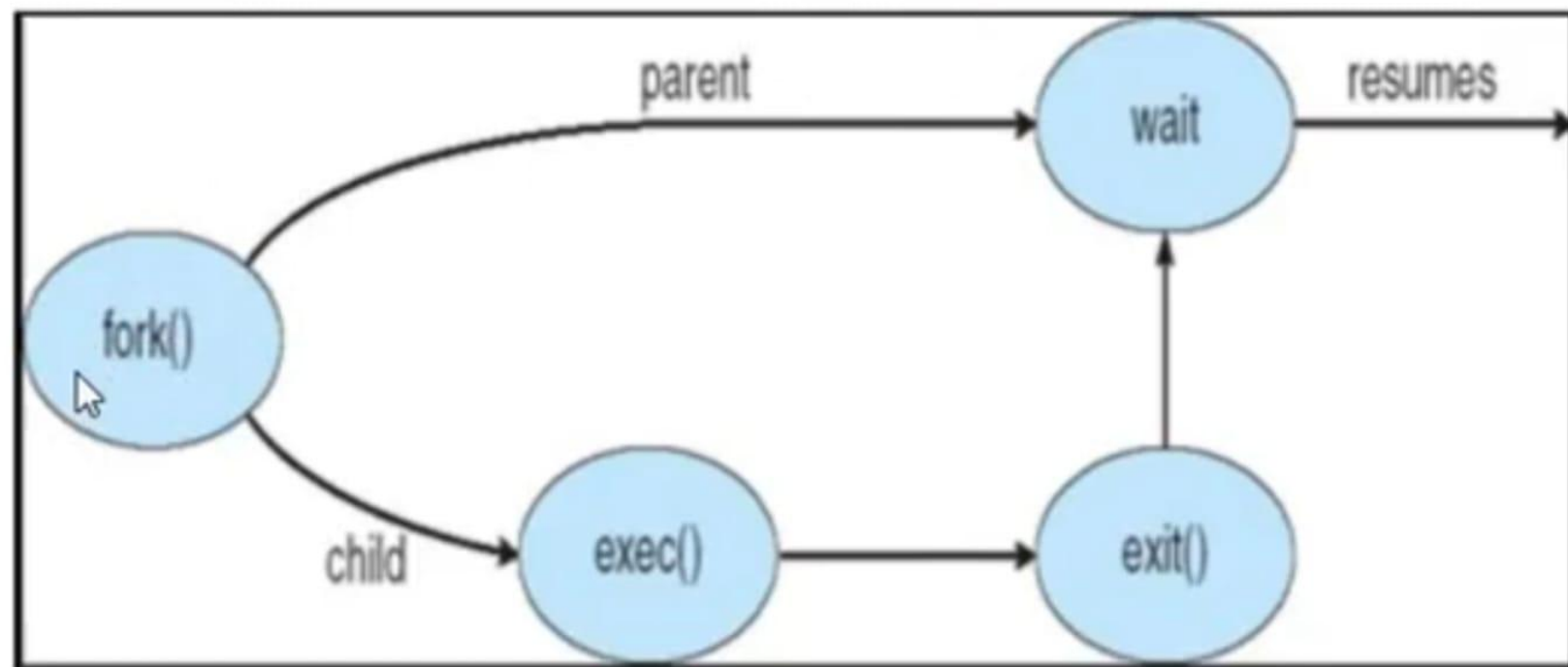
- A process may create several new processes. The creating process is called a **parent process**, and the new processes are called the **children** of that process.
- Each of these new processes may in turn create other processes, forming a **tree** of processes.
- Processes are assigned a unique process identifier (or **pid**), which is typically an integer number.

When a process creates a new process, two possibilities for execution exist:

1. The parent continues to execute **concurrently** with its children.
2. The parent **waits** until some or all of its children have terminated.

There are also two address-space possibilities for the new process:

1. The child process is a **duplicate** of the parent process (it has the same program and data as the parent).
2. The child process has a **new program** loaded into it.



Process Termination/ Deletion:

- A process terminates when it **finishes executing** its final statement and asks the operating system to delete it by using the `exit()` system call.
- At that point, the process may return a **status value** (typically an integer) to its parent process (via the `wait()` system call).
- All the resources of the process—including physical and virtual memory, open files, and I/O buffers—are **deallocated** by the operating system.
- A process can cause the **termination of another process** via an appropriate system call (for example, `TerminateProcess()` in Windows).

A parent may terminate the execution of one of its children for following reasons:

- The child has **exceeded its usage** of some of the resources that it has been allocated.
- The task assigned to the child is **no longer required**.
- The **parent is exiting**, and the operating system does not allow a child to continue if its parent terminates.
- If a process **terminates** (either normally or abnormally), then all its children must also be terminated. This phenomenon, referred to as **cascading termination**, is normally initiated by the operating system

Process Identification Information

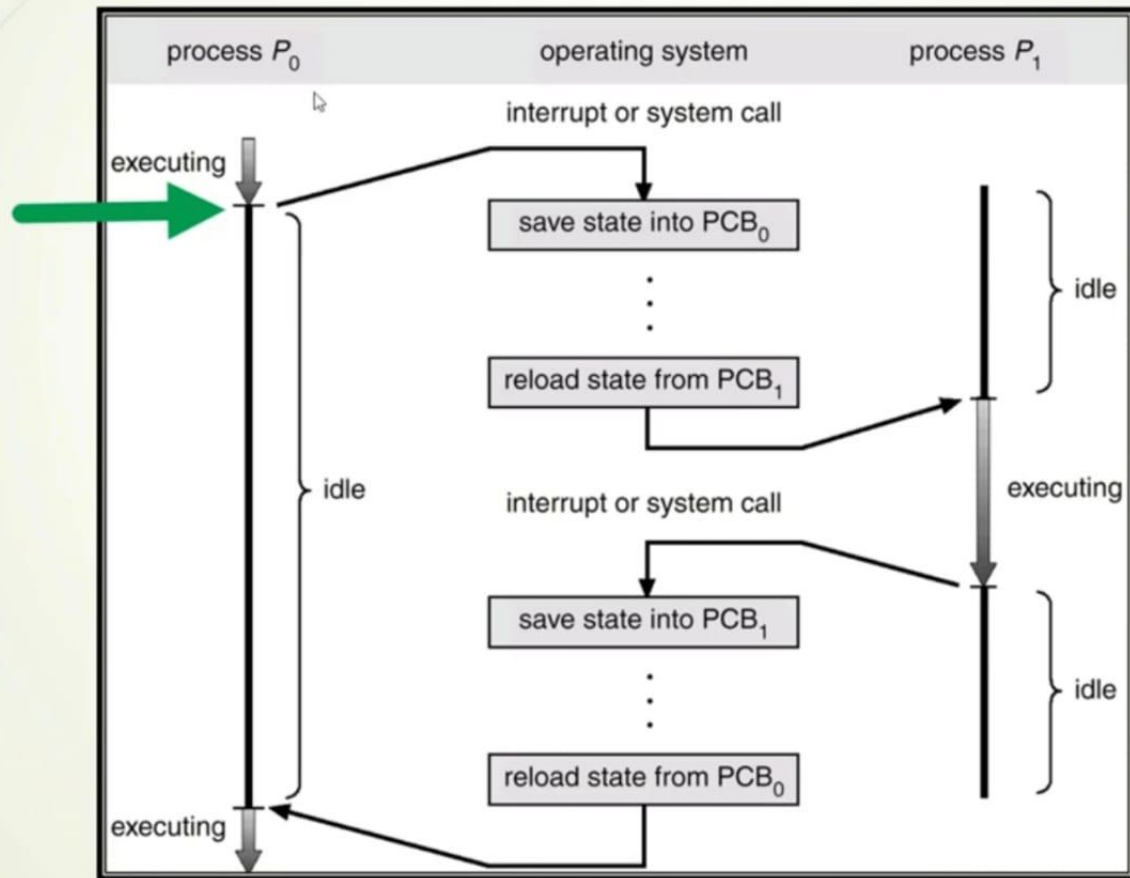
- **Process Identifier (process ID or PID)** is a number used by most operating system kernels **to uniquely identify** a process.
- This number may be used as a parameter in various function calls to **manipulate**, adjusting the process's **priority** or **killing** it.
- In Unix-like operating systems, new processes are created by the **fork()** system call. The parent may, for example, wait for the child to terminate with the **waitpid()** function, or terminate the process with **kill()**.

- Modern virtual memory operating systems generally have a **flat memory model** and not a segmented one.
- A **memory address** is a given value within the address space, such as 4021f000. The process can access a memory address only in a valid memory area.
- Memory areas have associated permissions, such as **readable, writable, and executable**.

- If a process accesses a memory address not in a valid memory area, or if it accesses a valid area in an invalid manner, the kernel kills the process with the "**Segmentation Fault**" message.



CPU Switch From Process to Process



Processes Vs Threads

	Process	Thread
Definition	Process means any program is in execution.	Thread means segment of a process.
Context switching time	Processes require more time for context switching as they are more heavy.	Threads require less time for context switching as they are lighter than processes.
Memory Sharing	Processes are totally independent and don't share memory.	A thread may share some memory with its peer threads.
Communication	Communication between processes requires more time than between threads.	Communication between threads requires less time than between processes .
Blocked	If a process gets blocked, remaining processes can continue execution.	If a user level thread gets blocked, all of its peer threads also get blocked.

Processes Vs Threads

	Process	Thread
Dependency	Individual processes are independent of each other.	Threads are parts of a process and so are dependent.
Data and Code sharing	Processes have independent data and code segments.	A thread shares the data segment, code segment, files etc. with its peer threads.
Time for creation	Processes require more time for creation.	Threads require less time for creation.
Time for termination	Processes require more time for termination.	Threads require less time for termination.

Context Switch

- Interrupts cause the operating system to change a CPU from its current task and to run a kernel routine.
- Such operations happen frequently on general-purpose systems.

When an interrupt occurs, the system needs to save the current **context** of the process currently running on the CPU so that it can restore that context when its processing is done, essentially suspending the process and then resuming it.

- The context is represented in the PCB of the process



Process Control Block

Switching the CPU to another process requires performing a state save of the current process and a state restore of a different process.



State Save



State Restore

This task is known as a **context switch**.

- Context-switch time is pure overhead, because the system does no useful work while switching.
- Its speed varies from machine to machine, depending on the memory speed, the number of registers that must be copied, and the existence of special instructions (such as a single instruction to load or store all registers).
- Typical speeds are a few milliseconds.

Process Scheduling

- The objective of multiprogramming is to have some process running at all times, to maximize CPU utilization.
- The objective of time sharing is to switch the CPU among processes so frequently that users can interact with each program while it is running.
- To meet these objectives, the process scheduler selects an available process (possibly from a set of several available processes) for program execution on the CPU.
 - For a single-processor system, there will never be more than one running process.
 - If there are more processes, the rest will have to wait until the CPU is free and can be rescheduled.



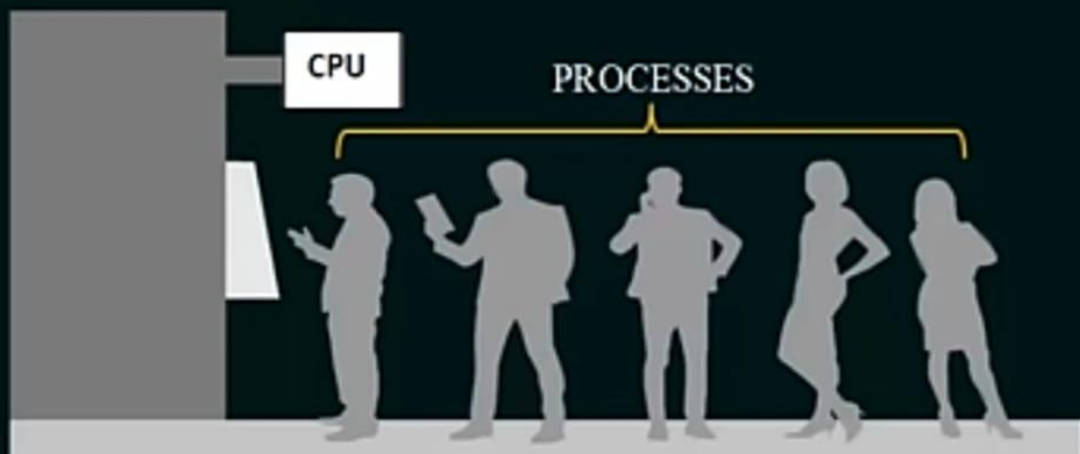
Scheduling Queues

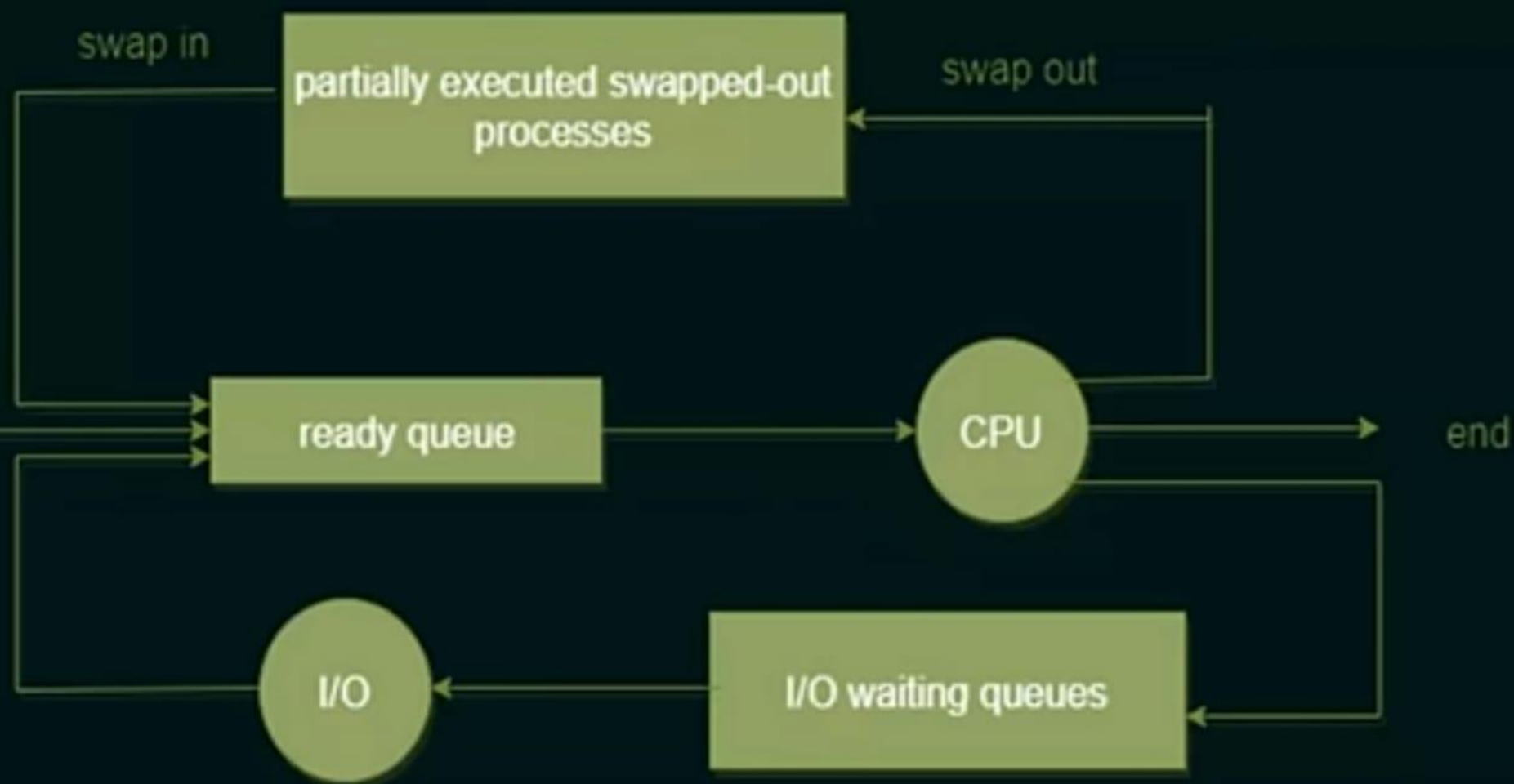
JOB QUEUE

As processes enter the system, they are put into a **job queue**, which consists of all processes in the system.

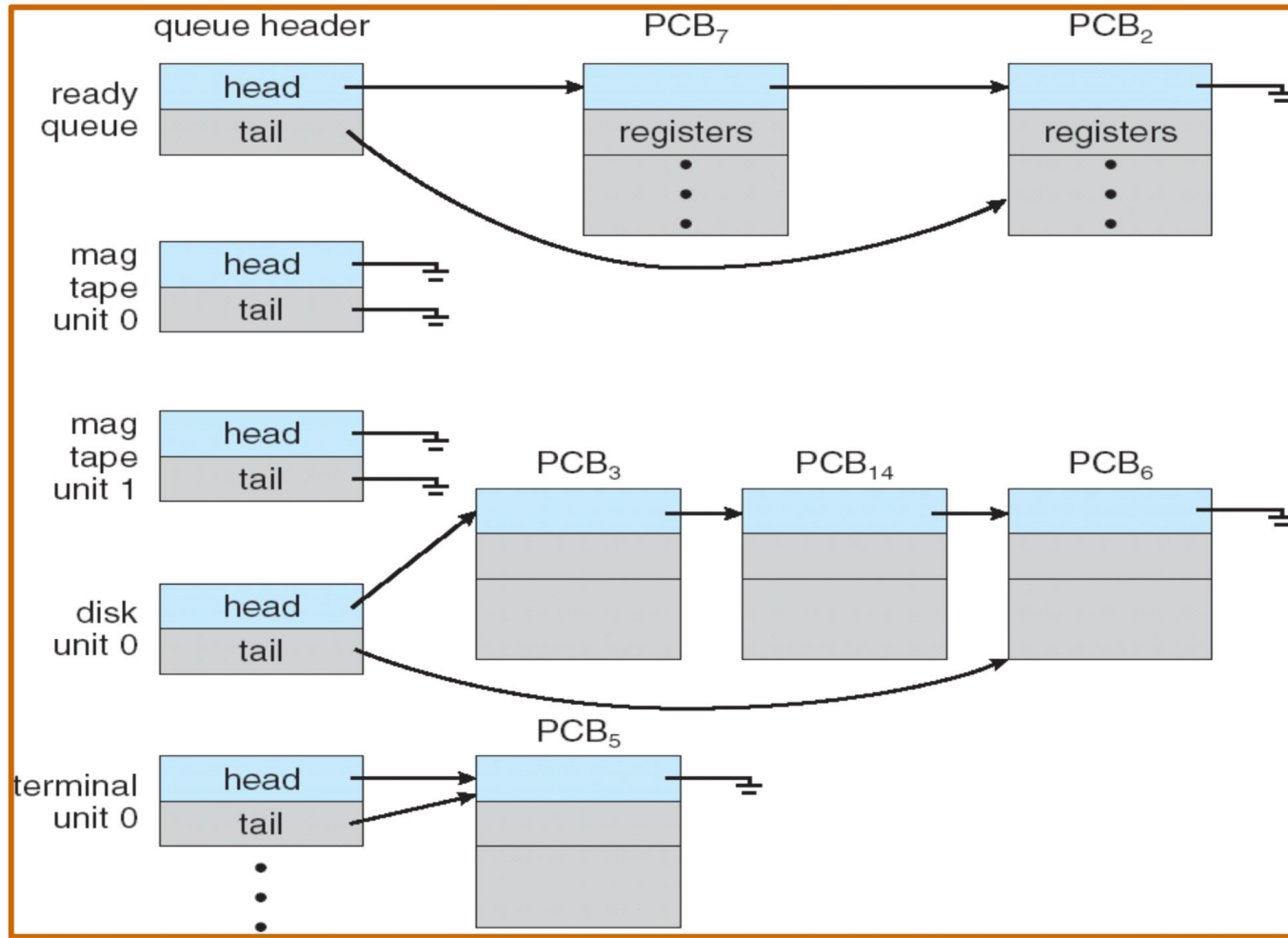
READY QUEUE

The processes that are residing in main memory and are ready and waiting to execute are kept on a list called the **ready queue**.

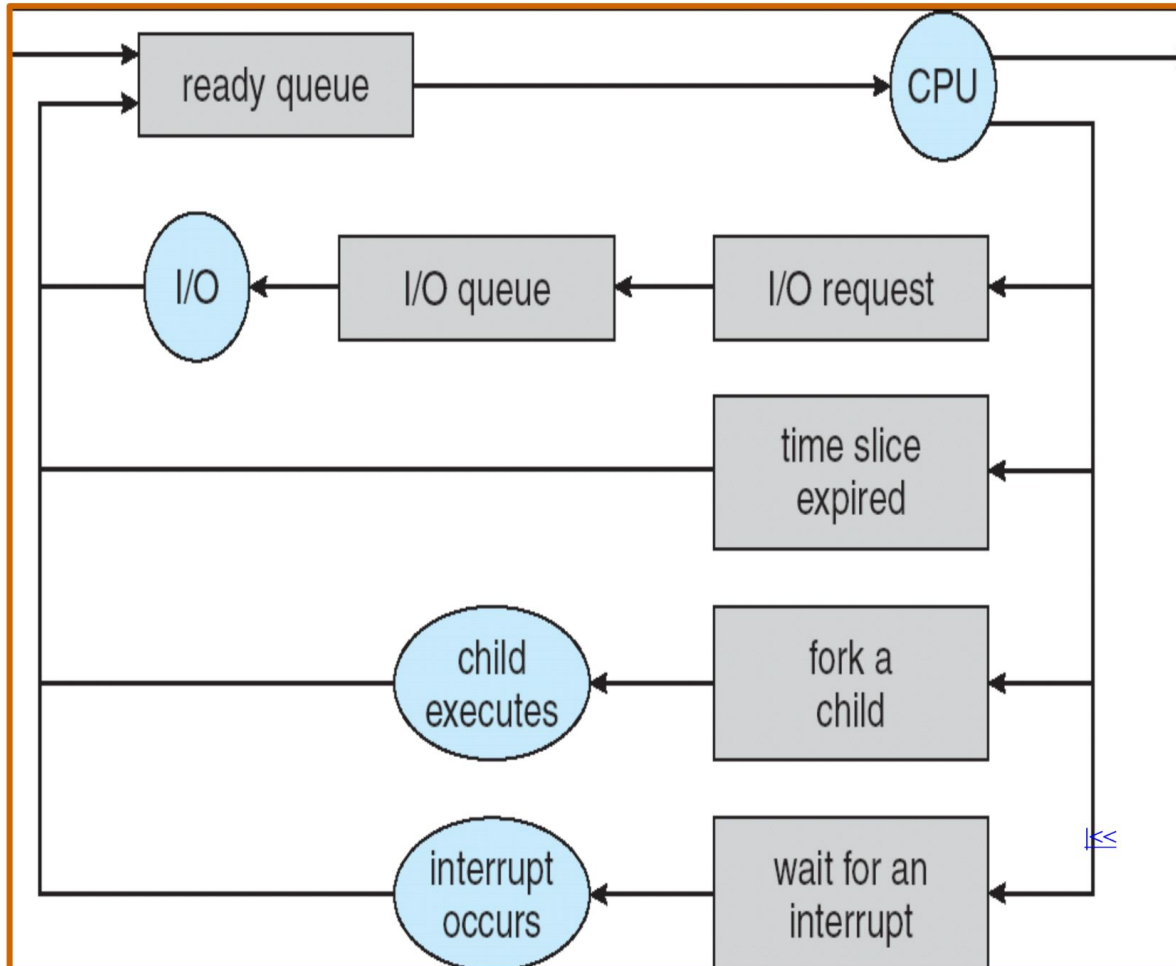




Ready Queue and Various I/O Device Queues



Representation of Process Scheduling



Schedulers: A process migrates among the various scheduling queues throughout its lifetime. The operating system must select, for scheduling purposes, processes from these queues in some fashion. The selection process is carried out by the appropriate **scheduler**. Schedulers have two types:

1. Long Term Scheduler

A. selects which process should be brought into the **ready queue**.

B. L.T.S. is invoked very infrequently (seconds, minutes).

C. L.T.S. controls the degree of multiprogramming (number of processes in memory)

2. Short Term Scheduler

A. selects which process should be executed next & allocates CPU.

B. STS is invoked very frequently (milliseconds).

Medium-Term Scheduler:

Medium-Term Scheduler removes processes from main memory and stores to secondary memory. Later, the process can be reintroduced into main memory, and its execution can be continued where it left off. This scheme is called **swapping**.

Dispatcher:

- A dispatcher is a special program which comes into play after the scheduler.
- When the scheduler completes its job of selecting a process, it is the dispatcher which takes that process to the desired state/queue.
- The dispatcher is the module that gives a process control over the CPU after it has been selected by the short-term scheduler.

This function involves:

● Switching Context

- When CPU switches to another process, the system must save the state of the old process and load the saved state for the new process.

- Switching to user mode
- Jumping to the proper location in the user program to restart that program.



The dispatcher should be as fast as possible, given that it is invoked during every process switch. The time it takes for the dispatcher to stop one process and start another running is known as **dispatch latency**.

Preemptive and Non-Preemptive Scheduling

CPU Scheduler

Whenever the CPU becomes idle, the operating system must select one of the processes in the ready queue to be executed. The selection process is carried out by the short-term scheduler (or CPU scheduler). The scheduler selects a process from the processes in memory that are ready to execute and allocates the CPU to that process.

Dispatcher

The dispatcher is the module that gives control of the CPU to the process selected by the short-term scheduler.

The time it takes for the dispatcher to stop one process and start another running is known as the **dispatch latency**.

CPU-scheduling decisions may take place under the following four circumstances:

1. When a process switches from the **running state** to the **waiting state**
2. When a process switches from the **running state** to the **ready state** (for example, when an interrupt occurs).
3. When a process switches from the **waiting state** to the **ready state** (for example, at completion of I/O).
4. When a process terminates.

For situations 1 and 4, there is no choice in terms of scheduling. A new process (if one exists in the ready queue) must be selected for execution.

However, there is a choice for situations 2 and 3.

When scheduling takes place only under circumstances 1 and 4, we say that the scheduling scheme is nonpreemptive or cooperative; otherwise, it is preemptive.

Preemptive Scheduling



CPU is allocated to a process for limited time.

Resource Allocation

Process can be interrupted in between

Interrupt

Low priority process may starve

Starvation

Switching the processes frequently increases the overhead

Overhead

Non-Preemptive Scheduling



CPU is allocated to a process till it complete burst time or switches to waiting state.

Process can not be interrupted in between

Process with short burst time may starve

Doesn't have switching overhead

Preemptive Scheduling

```
graph LR; PS[Preemptive Scheduling] --> F1[Preemptive scheduling is flexible]; PS --> C1[Cost associated]; PS --> E1[Round Robin, Priority Scheduling, Shortest Remaining Time First, etc.]; NPS[Non-Preemptive Scheduling] --> F2[Non-Preemptive scheduling is rigid]; NPS --> C2[No cost associated]; NPS --> E2[First Come First serve, Shortest Job First, etc.]; F1 <--> F3[Flexibility]; C1 <--> C3[Cost]; E1 <--> E3[Example];
```

Preemptive scheduling is flexible

Flexibility

Non-Preemptive scheduling is rigid

Cost associated

Cost

No cost associated

Round Robin, Priority Scheduling, Shortest Remaining Time First, etc.

Example

First Come First serve, Shortest Job First, etc.

Scheduling Performance Criteria:



- **CPU Utilization:** We want to keep the CPU as busy as possible. Conceptually, CPU utilization can range from 0 to 100 percent.

$$\text{Processor Utilization} = (\text{Processor Busy Time} / (\text{Processor Busy Time} + \text{Processor Idle time})) * 100$$

- **Throughput:** the number of processes that are completed per time unit, called throughput.

$$\text{Throughput} = \text{No. of Process Completed} / \text{Time Unit}$$

- **Turnaround Time:** The amount of time to execute a particular process is called turnaround time.

$$\text{Turnaround Time} = T(\text{Process Completed}) - T(\text{Process Submitted})$$

time.

$$\text{Turnaround Time} = T(\text{Process Completed}) - T(\text{Process Submitted})$$

- Waiting Time: the amount of time that a process spends waiting in the ready queue.

$$\text{Waiting Time} = \text{Turnaround Time} - \text{Processing Time}$$

- Response Time: time from the submission of a request until the first response is produced.
This measure, called response time.

$$\text{Response Time} = T(\text{First Response}) - T(\text{submission of request})$$

➤ Optimization Criteria:

- ✓ Max CPU utilization
- ✓ Max throughput
- ✓ Min turnaround time
- ✓ Min waiting time
- ✓ Min response time

Scheduling Algorithms:

1. First-Come, First-Served Scheduling
2. Shortest-Job-First Scheduling
3. Priority Scheduling
4. Round-Robin Scheduling
5. Multilevel Queue Scheduling
6. Multilevel Feedback Queue Scheduling

f CF

FIRST-COME, FIRST-SERVED SCHEDULING

Example:

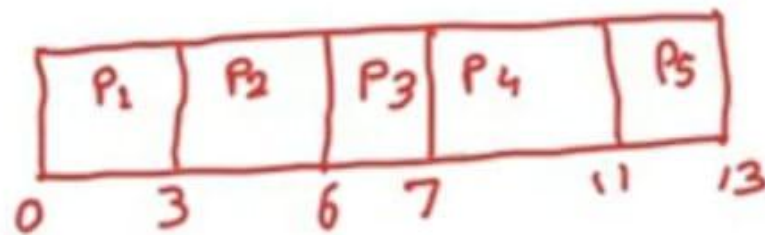
Process **p1,p2,p3,p4,p5** having arrival time of **0,2,3,5,8** microseconds and processing time **3,3,1,4,2** microseconds, Draw Gantt Chart & Calculate Average Turn Around Time, Average Waiting Time, CPU Utilization & Throughput using **FCFS**.

Processes	Arrival Time	Processing Time	T.A.T. $T(P.C.) - T(P.S.)$	W.T. $TAT - T(Proc.)$
P1	0	3		
P2	2	3		
P3	3	1		
P4	5	4		
P5	8	2		

Processes	Arrival Time	Processing Time	T.A.T. T(P.C.)-T(P.S.)	W.T. TAT- T(Proc.)
P1 ✓	0 ✓	3 ✓	$3 - 0 = 3$	$3 - 3 = 0$
P2 ✓	2 ✓	3 ✓	$6 - 2 = 4$	$4 - 3 = 1$
P3 ✓	3 ✓	1 ✓	$7 - 3 = 4$	$4 - 1 = 3$
P4 ✓	5 ✓	4 ✓	$11 - 5 = 6$	$6 - 4 = 2$
P5 ✓	8 ✓	2 ✓	$13 - 8 = 5$	$5 - 2 = 3$

P2 ✓
 P3 ✓
 P4 ✓
 P5 ✓

GANTT CHART:

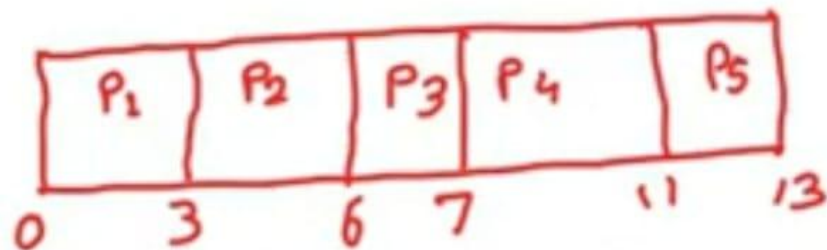


$$\text{Average T.A.T.} = \frac{3 + 4 + 4 + 6 + 5}{5} = \frac{22}{5} = 4.4 \text{ msec.}$$

$$\text{Average W.T.} = \frac{0 + 1 + 3 + 2 + 3}{5} = \frac{9}{5} = 1.8 \text{ msec.}$$

P3 ✓
P4 ✓
P5 ✓

GANTT CHART:



Average T.A.T. = $\frac{3+4+4+6+5}{5} = \frac{22}{5} = 4.4 \mu\text{sec} \checkmark$

Average W.T. = $\frac{0+1+3+2+2}{5} = \frac{9}{5} = 1.8 \mu\text{sec} \checkmark$

CPU Utilization = $\frac{13}{13+0} * 100 = 100\% \checkmark$

Throughput = $5/13 = 0.38 \checkmark$

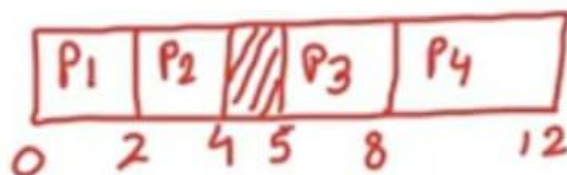
Process **p1,p2,p3,p4** having arrival time of **0,1,5,6** microseconds and burst time **2,2,3,4** microseconds, Draw Gantt Chart & Calculate Average Turn Around Time, Average Waiting Time, Average Response Time, CPU Utilization & Throughput using **FCFS**.

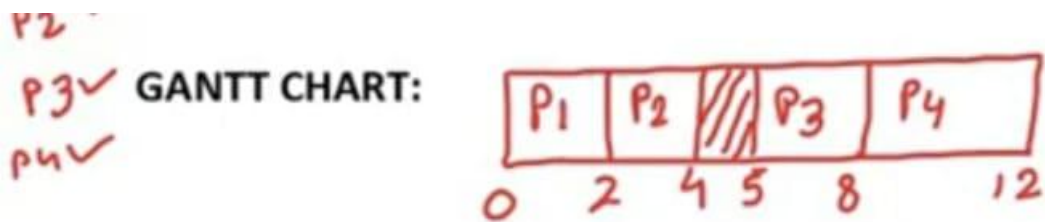
Processes	Arrival Time	Processing Time	T.A.T. T(P.C.)-T(P.S.)	W.T. TAT- T(Proc.)
P1	0	2		
P2	1	2		
P3	5	3		
P4	6	4		

Process **p1,p2,p3,p4** having arrival time of 0,1,5,6 microseconds and burst time 2,2,3,4 microseconds, Draw Gantt Chart & Calculate Average Turn Around Time, Average Waiting Time, Average Response Time, CPU Utilization & Throughput using FCFS.

Processes	Arrival Time	Burst Processing Time	T.A.T. $T(P.C.) - T(P.S.)$	W.T. $TAT - T(Proc.)$	RT $T(FR) - T(P.S.)$
P1	0	2 -	$2 - 0 = 2$	$2 - 2 = 0$	$0 - 0 = 0$
P2	1	2 -	$4 - 1 = 3$	$3 - 2 = 1$	$2 - 1 = 1$
P3	5	3 -	$8 - 5 = 3$	$3 - 3 = 0$	$5 - 5 = 0$
P4	6	4 -	$12 - 6 = 6$	$6 - 4 = 2$	$8 - 6 = 2$

GANTT CHART:





$$\frac{0+1+0+2}{4} = \frac{3}{4}$$

$$= 0.75 \mu sec.$$

Average T.A.T. = $\frac{2+2+3+6}{4} = \frac{14}{4} = 3.5 \mu sec.$

Busy. —
4 + 3 + 4 = 11 msec.

Average W.T. = $\frac{0+1+0+2}{4} = \frac{3}{4} = 0.75 \mu sec.$

Idle = 1 msec.

CPU Utilization = $\frac{11}{11+1} * 100 = \frac{11}{12} * 100 = \frac{550}{6} = 91.67\%$

Throughput = $\frac{4}{12} = 0.33$

Shortest-Job-First Scheduling (SJF):

- Associate with each process the length of its next CPU burst. Use these lengths to schedule the process with the shortest time
- **Two schemes:**
 - i. **nonpreemptive** – once CPU given to the process it cannot be preempted until completes its CPU burst
 - ii. **preemptive** – if a new process arrives with CPU burst length less than remaining time of current executing process, preempt. **This scheme is known as the Shortest-Remaining-Time-First (SRTF)**
- **SJF is optimal** – gives minimum average waiting time for a given set of processes

Example:

Process **p1,p2,p3,p4** having burst time of **6,8,7,3** microseconds. Draw Gantt Chart & Calculate Average Turn Around Time, Average Waiting Time, CPU Utilization & Throughput using **SJF**.

Processes	Burst Time	T.A.T. $T(P.C.) - T(P.S.)$	W.T. $TAT - T(Proc.)$
P4	3		
P1	6		
P3	7		
P2	8		

GANTT CHART

Average T.A.T. =

Average W.T. =

CPU Utilization =

non

		T(P.C.)-T(P.S.)	TAT- T(Proc.)
P4 ✓	3 ✓	$3 - 0 = 3$	$3 - 3 = 0$
P1 ✓	6 ✓	$9 - 0 = 9$	$9 - 6 = 3$
P3 ✓	7 ✓	$16 - 0 = 16$	$16 - 7 = 9$
P2 ✓	8 ✓	$24 - 0 = 24$	$24 - 8 = 16$

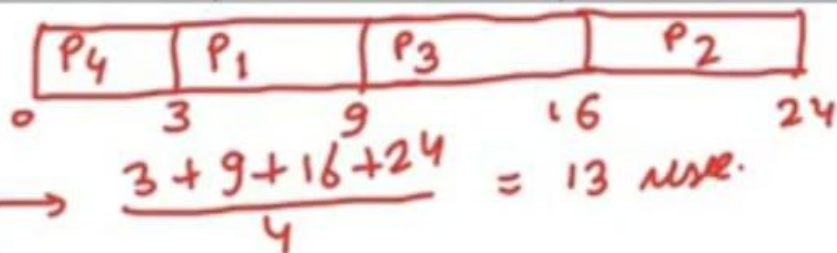
GANTT CHART

Average T.A.T. =

Average W.T. =

CPU Utilization =

Throughput =



$$\frac{0 + 3 + 9 + 16}{4} = \frac{28}{4} = 7 \text{ use.}$$

$$\frac{24}{24 + 0} \times 100 = 100\%$$

$$4/24 = ?$$

Process p1, p2, p3, p4 having arrival time of 0, 2, 4, 5 microseconds and burst time 7, 4, 1, 4 microseconds, Draw Gantt Chart & Compare Average Waiting Time using preemptive & non preemptive SJF.

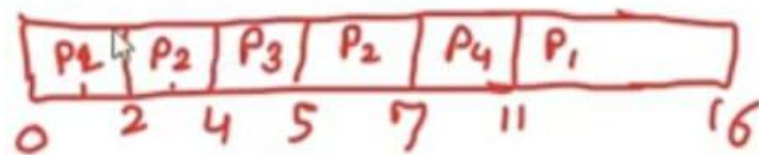
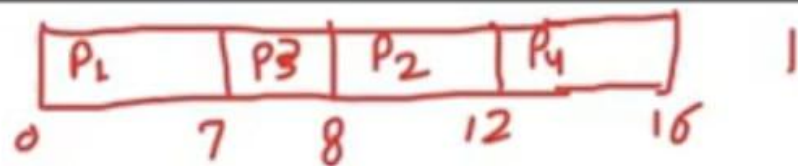
$P_1 = 5, P_2 = 4, P_3 = 1, P_4 = 4$

→ Non-Preemptive SJF

Processes	Arrival Time	Processing Time	T.A.T. T(P.C.)- T(P.S.)	W.T. TAT- T(Proc.)
P1	0	7		
P2	2	4		
P3	4	1		
P4	5	4		

→ Preemptive SJF

Processes	Arrival Time	Processing Time	T.A.T. T(P.C.)- T(P.S.)	W.T. TAT- T(Proc.)
P1	0	7		
P2	2	4		
P3	4	1		
P4	5	4		



Process p1, p2, p3, p4 having arrival time of 0, 2, 4, 5 microseconds and burst time 7, 4, 1, 4 microseconds, Draw Gantt Chart & Compare Average Waiting Time using preemptive & non preemptive SJF.

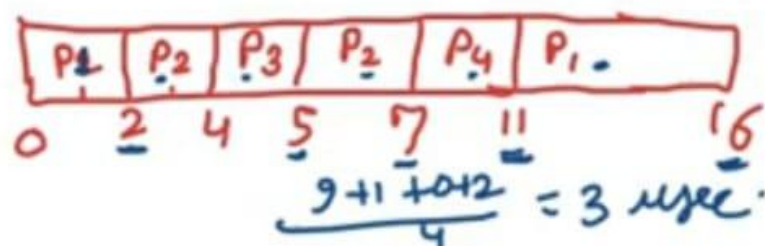
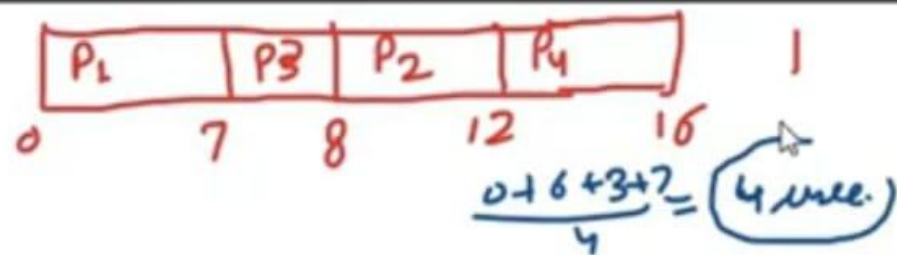
P1 = 5, ~~P2 = 4~~, ~~P3 = 1~~, ~~P4 = 4~~

→ Non-Preemptive SJF

Processes	Arrival Time	Processing Time	T.A.T. T(P.C.)- T(P.S.)	W.T. TAT- T(Proc.)
P1	0	7	$7 - 0 = 7$	$7 - 7 = 0$
P2	2	4	$12 - 2 = 10$	$10 - 4 = 6$
P3	4	1	$8 - 4 = 4$	$4 - 1 = 3$
P4	5	4	$16 - 5 = 11$	$11 - 4 = 7$

→ Preemptive SJF

Processes	Arrival Time	Processing Time	T.A.T. T(P.C.)- T(P.S.)	W.T. TAT- T(Proc.)
P1	0	7	$16 - 0 = 16$	$16 - 7 = 9$
P2	2	4	$7 - 2 = 5$	$5 - 4 = 1$
P3	4	1	$5 - 4 = 1$	$1 - 1 = 0$
P4	5	4	$11 - 5 = 6$	$6 - 4 = 2$



1. SJF non-preemptive

Proc

Arrives

Burst

P1	0	8
P2	1	4
P3	2	9
P4	3	5

And then preemptive

2. SJF non-preemptive

Proc

Arrives

Burst

P1	1	2
P2	0	7
P3	2	7
P4	5	3
P5	6	1

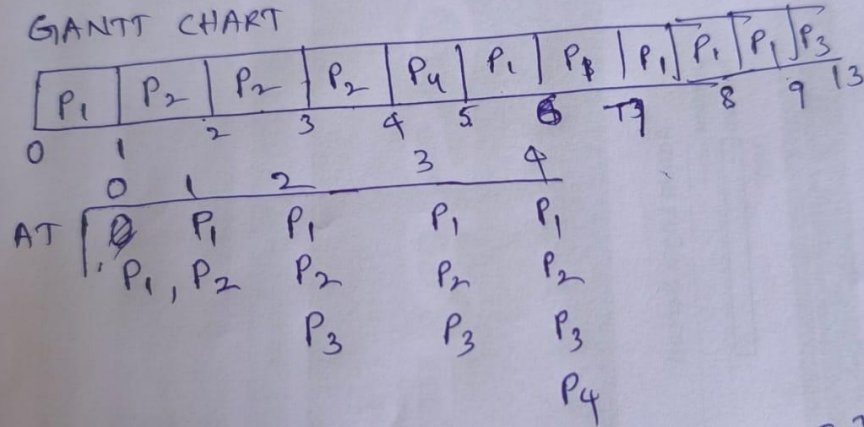
And then preemptive

Shortest Remaining Time First (SRTF)

PS.NO	A.T	B.T	C.T	TAT CT-AT	WT TAT-BT	RT CT-First time-AT
P ₁	0	8	9	9-0=9	9-8=1	0-0=0
P ₂	1	3	4	4-1=3	3-3=0	1-1=0
P ₃	2	4	13	13-2=11	11-4=7	9-2=7
P ₄	4	1	5	5-4=1	1-1=0	4-4=0

Also called SJF with preemption.

GANTT CHART



$$AVG TAT = \frac{24}{4} = 6$$

$$AVG WT = \frac{11}{4} = 2.75$$

$$AVG RT = \frac{7}{4} = 1.75$$

Priority scheduling: (preemptive)

Higher the number higher the priority

Priority	Process no	A.T	B.T	C.T	TAT CT-AT	WT TAT-BT
10	P ₁	0	8 40	12	12-0=12	12-5=7
20	P ₂	1	4 30	8	8-1=7	7-4=3
30	P ₃	2	2 10	4	4-2=2	2-2=0
40	P ₄	4	2 0	5	5-4=1	1-1=0

AT	0	1	2	3	4
	P ₁	P ₁ P ₂	P ₁ P ₂ P ₃	P ₁ P ₂ P ₃	P ₁ P ₂ P ₃ P ₄

P_1	P_2	P_3	P_3	P_4	P_2	P_1	
0	1	2	3	4	5	8	12

$$AVG TAT = \frac{22}{4} = 5.5 \quad AVG WT = \frac{10}{4} = 2.5$$

Example

Proc	Arrival	Burst	Priority
------	---------	-------	----------

P1	0	6	5
----	---	---	---

P2	2	2	3
----	---	---	---

P3	3	3	4
----	---	---	---

P4	9	3	2
----	---	---	---

P5	10	1	1
----	----	---	---

Round-Robin Scheduling:

- Each process gets a small unit of CPU time (*time quantum*), usually 10-100 milliseconds. After this time has elapsed, the process is preempted and added to the end of the ready queue.
- Used for time sharing & multiuser O.S.
- FCFS with preemptive scheduling.

Example:

Process **p1,p2,p3** having processing time of **24,3,3** milliseconds. Draw Gantt Chart & Calculate Average Turn Around Time, Average Waiting Time, CPU Utilization & Throughput using **Round Robin** with time slice of 4milliseconds.

Processes	Processing Time	T.A.T. T(P.C.)-T(P.S.)	W.T. TAT- T(Proc.)
P1	24		
P2	3		
P3	3		

GANTT CHART**Average T.A.T. =**

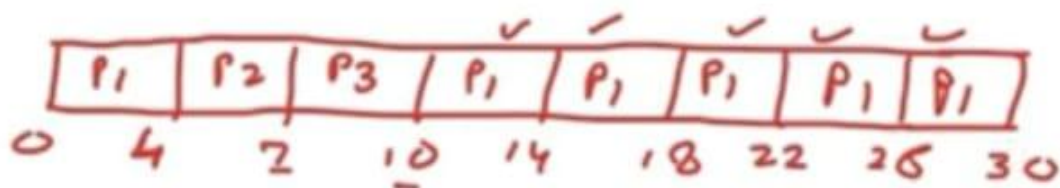
Example:

Process p1, p2, p3 having processing time of 24, 3, 3 milliseconds. Draw Gantt Chart & Calculate Average Turn Around Time, Average Waiting Time, CPU Utilization & Throughput using Round Robin with time slice of 4 milliseconds.

$$T.Q. = 4$$

Processes	Processing Time	T.A.T. T(P.C.)-T(P.S.)	W.T. TAT- T(Proc.)
P1	24 -	30 - 0 = 30	
✓ P2	3 -	7 - 0 = 7	
✓ P3	3 -	10 - 0 = 10	

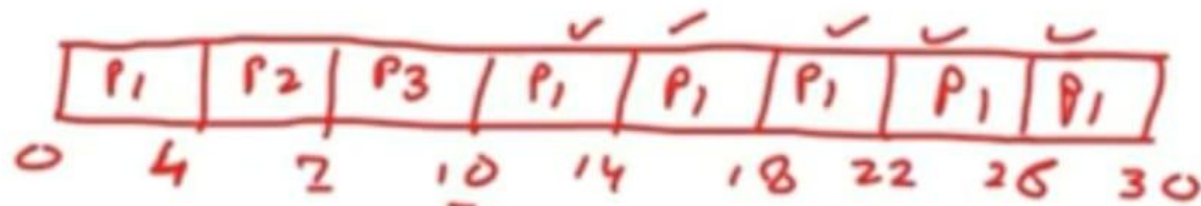
GANTT CHART



$$\text{Average T.A.T.} = \frac{30 + 7 + 10}{3} = \frac{47}{3} = 15.67 \text{ m.sec.}$$

Processes	Processing Time	T.A.T. T(P.C.)-T(P.S.)	W.T. TAT- T(Proc.)
P1	24 ✓	$30 - 0 = 30$	$30 - 24 = 6$
✓ P2	3 ✓	$7 - 0 = 7$	$7 - 3 = 4$
✓ P3	3 ✓	$10 - 0 = 10$	$10 - 3 = 7$

GANTT CHART



$$\text{Average T.A.T.} = \frac{30 + 7 + 10}{3} = \frac{47}{3} = 15.67 \text{ m. sec.}$$

$$\text{Average W.T.} = \frac{6 + 4 + 7}{3} = \frac{17}{3}$$

$$\text{Average T.A.T.} = \frac{30 + 7 + 10}{3} = \frac{47}{3} = 15.67 \text{ msec.}$$

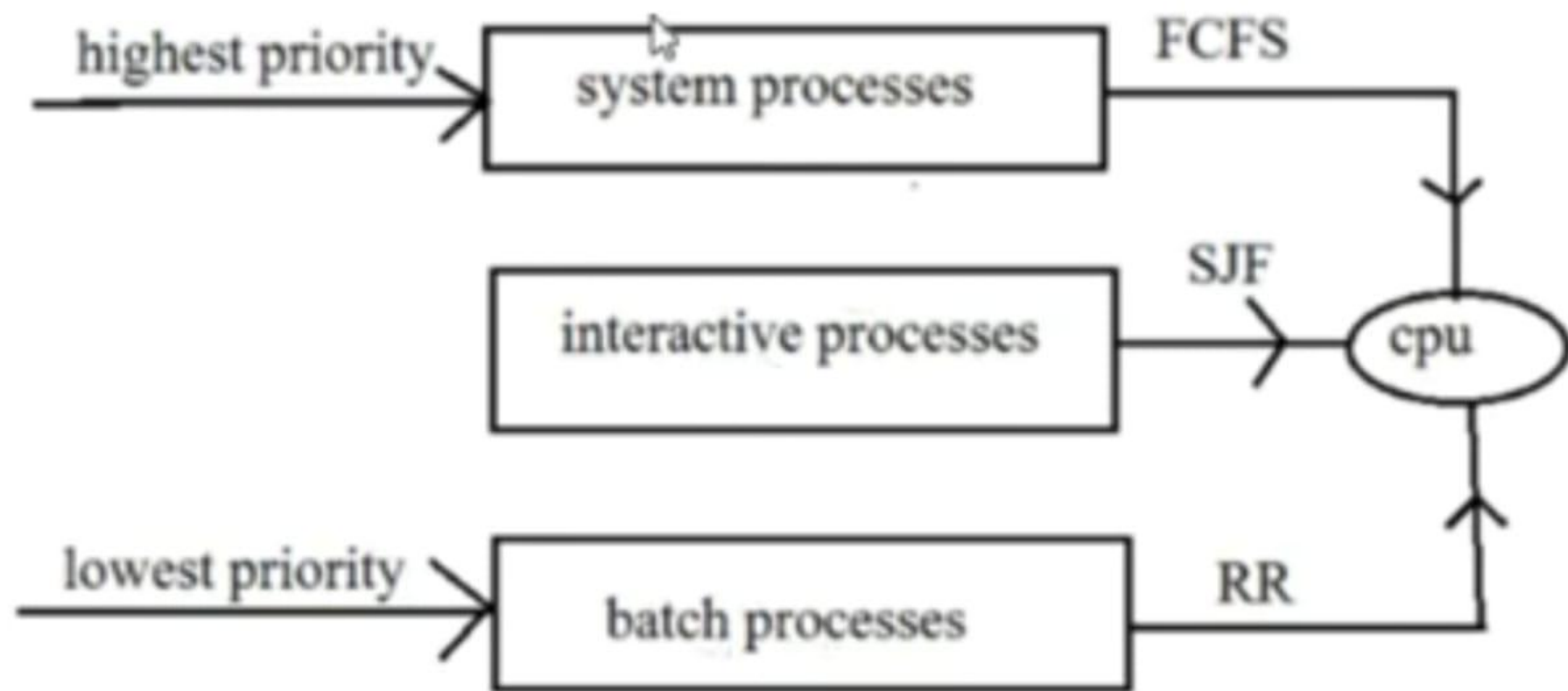
$$\text{Average W.T.} = \frac{6 + 4 + 7}{3} = \frac{17}{3} = 5.67 \text{ msec.}$$

$$\text{CPU Utilization} = \frac{30}{30 + 0} \times 100 = 100\%$$

$$\text{Throughput} = 3/30 = 0.1$$

Multilevel Queue Scheduling

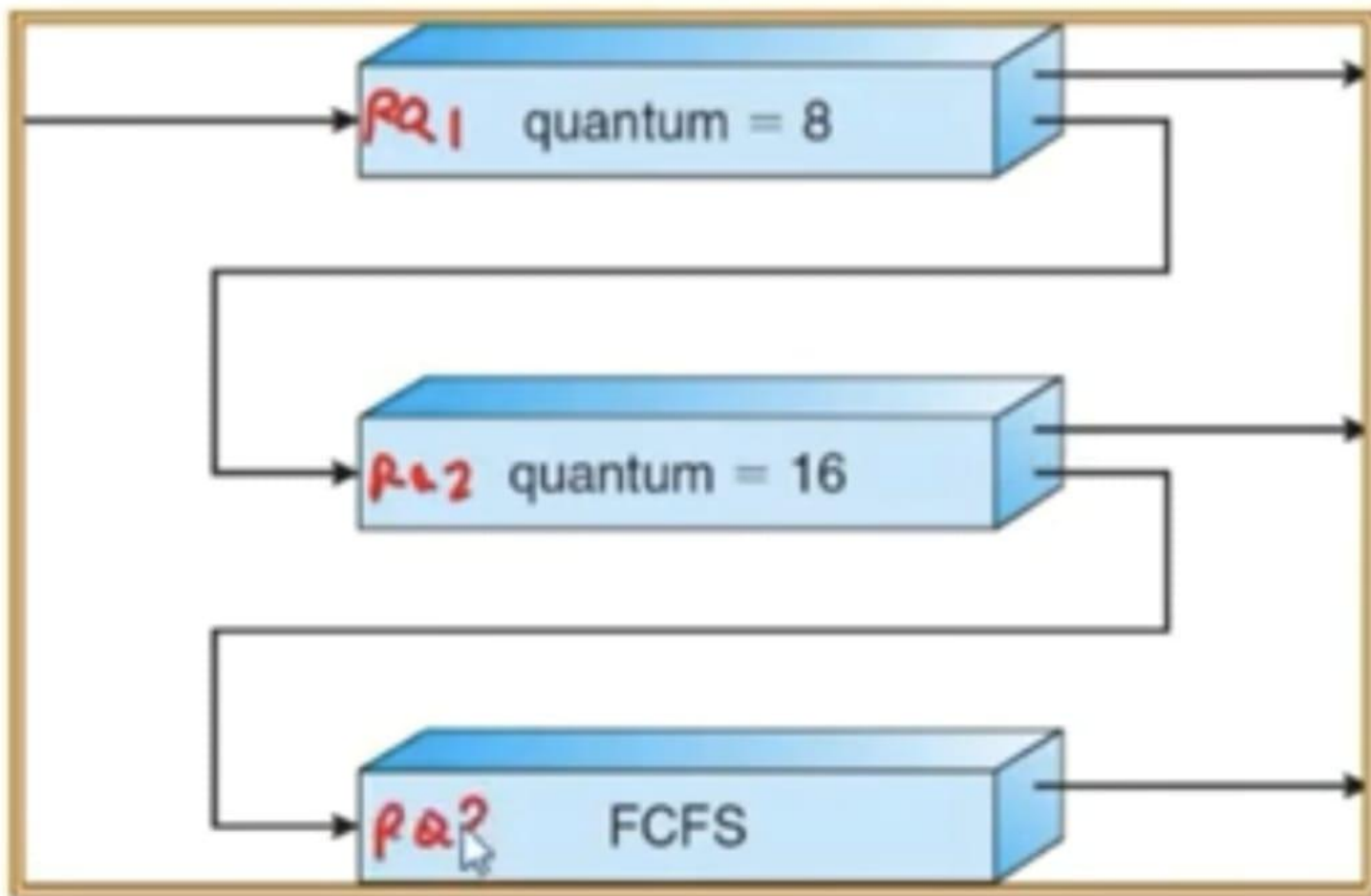
- A multi level queue scheduling algorithm partitions the ready queue into several separate queues.
- Each queue has its own scheduling algorithm.
- Advantage- We can apply various kind of scheduling for different kind of processes.
- Disadvantage- Problem of starvation for lower-level processes.



Multilevel Feedback Queue Scheduling

Processes are assigned to a queue on entry to the system, and processes are allowed to move between queues.

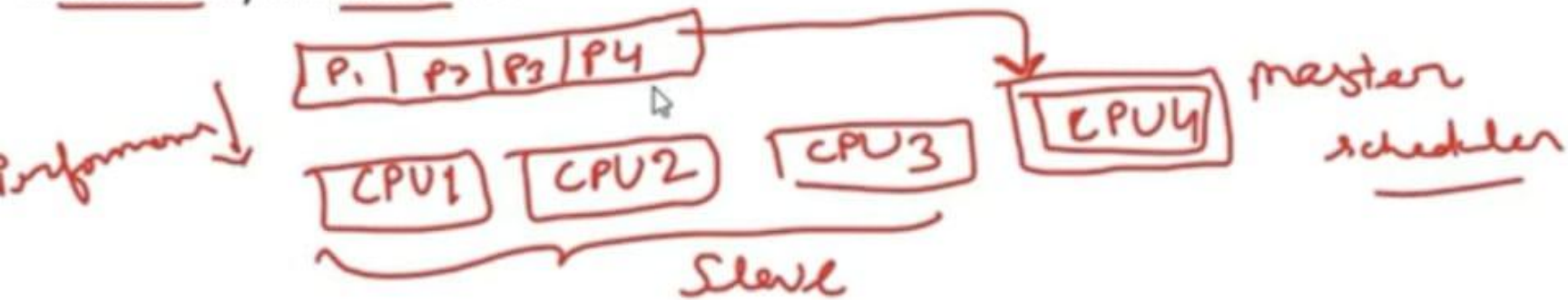
It prevents starvation by moving a process that waits too long for the lower priority queue to the higher priority queue.



Multiple Processor Scheduling

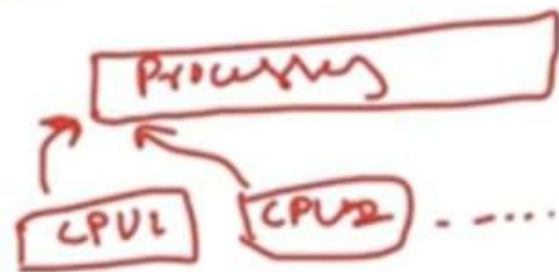
- In multiple-processor scheduling multiple CPU's are available and hence Load Sharing becomes possible.
- **Asymmetric Multiprocessing**- All the scheduling decisions and I/O processing are handled by a single processor which is called the **Master Server** and the other processors acts as **Slave** executes only the **user code**.

Asymmetric Multiprocessing- All the scheduling decisions and I/O processing are handled by a single processor which is called the Master Server and the other processors acts as Slave executes only the user code.



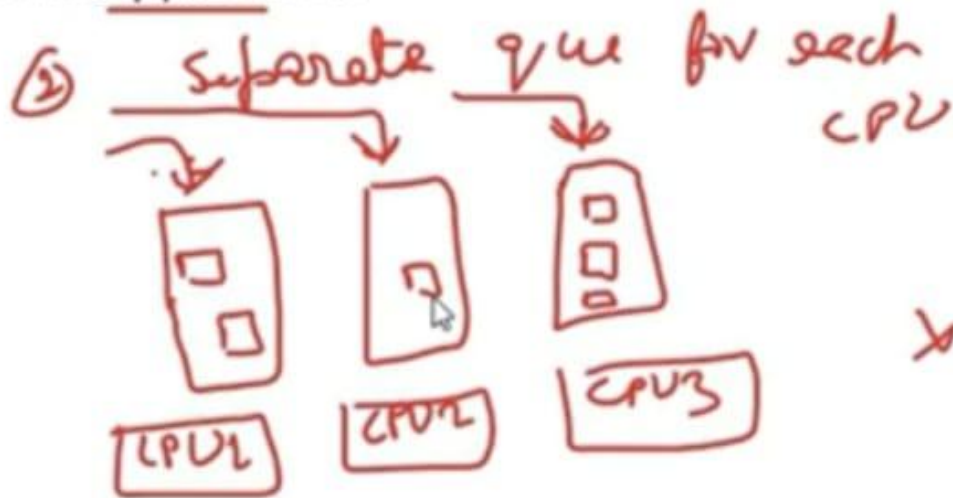
Symmetric Multiprocessing- All processes may be in a common ready queue or each processor may have its own private queue for ready processes.

① using global que



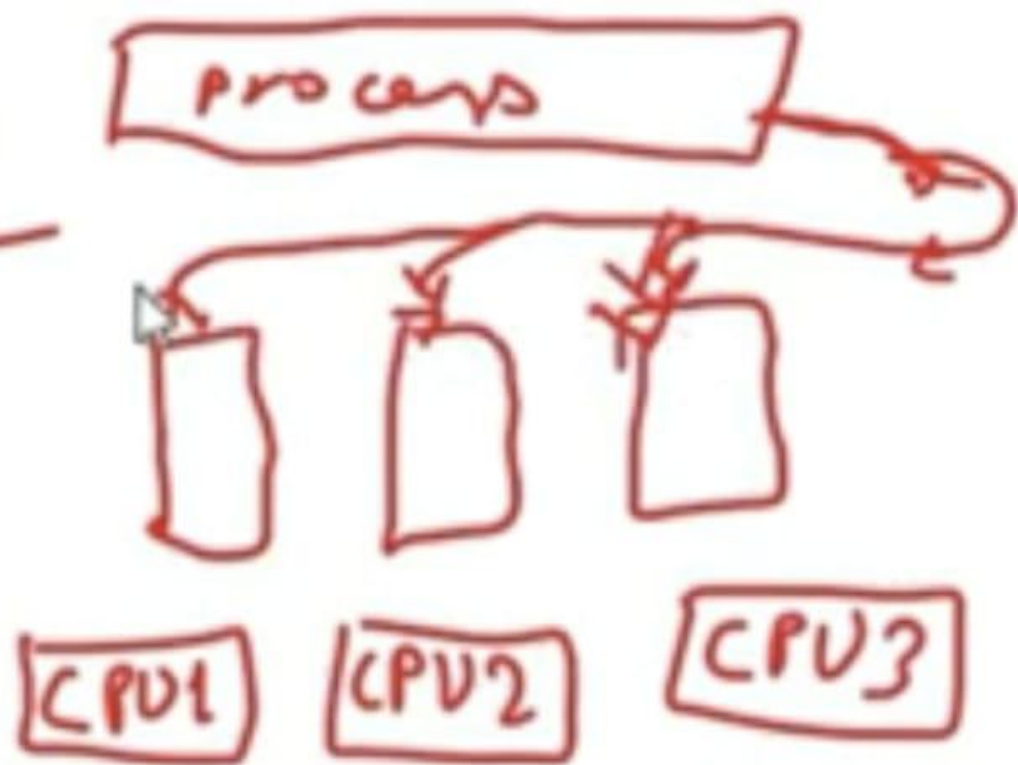
CPU utilization - Adv.

locking mechanism - wait - Disadv.



③

hybrid



Real Time Scheduling



- Real-time systems used to perform real-time tasks.
- RTOS scheduler is to meet task deadlines, instead of throughput, latency and response time, etc.
- **Hard real-time systems** – required to complete a critical task within a guaranteed amount of time.
- **Soft real-time computing** – requires that critical processes receive priority over less fortunate ones.

UNIT - III



Interprocess Communication:

- Processes executing concurrently in the operating system may be either **independent processes** or **cooperating processes**.
- A process is **independent** if it cannot affect or be affected by the other processes executing in the system. Any process that does not share data with any other process is independent.
- A process is **cooperating** if it can affect or be affected by the other processes executing in the system. Clearly, any process that shares data with other processes is a cooperating process.

There are several reasons for process cooperation:

- **Information sharing:** Since several users may be interested in the same piece of information (for instance, a shared file), we must provide an environment to allow concurrent access to such information.
- **Computation speedup:** If we want a particular task to run faster, we must break it into subtasks, each of which will be executing in parallel with the others. Such a speedup can be achieved only if the computer has multiple processing cores.

- **Modularity**: We may want to construct the system in a modular fashion, dividing the system functions into separate processes or threads.
- **Convenience**: Even an individual user may work on many tasks at the same time. For instance, a user may be editing, listening to music, and compiling in parallel.

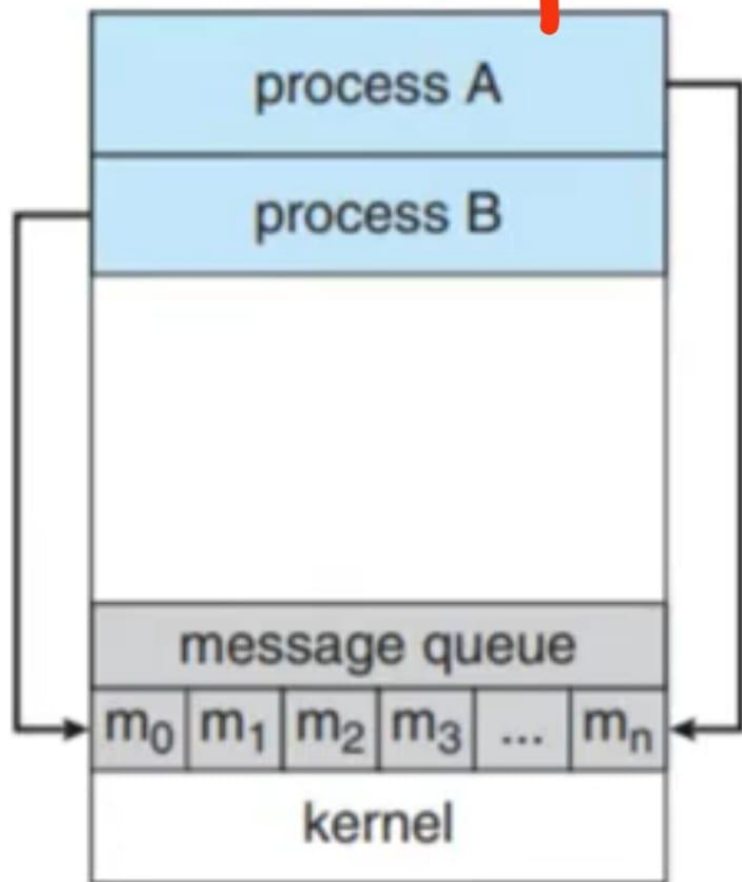
Cooperating processes require an interprocess communication (IPC) mechanism that will allow them to exchange data and information.



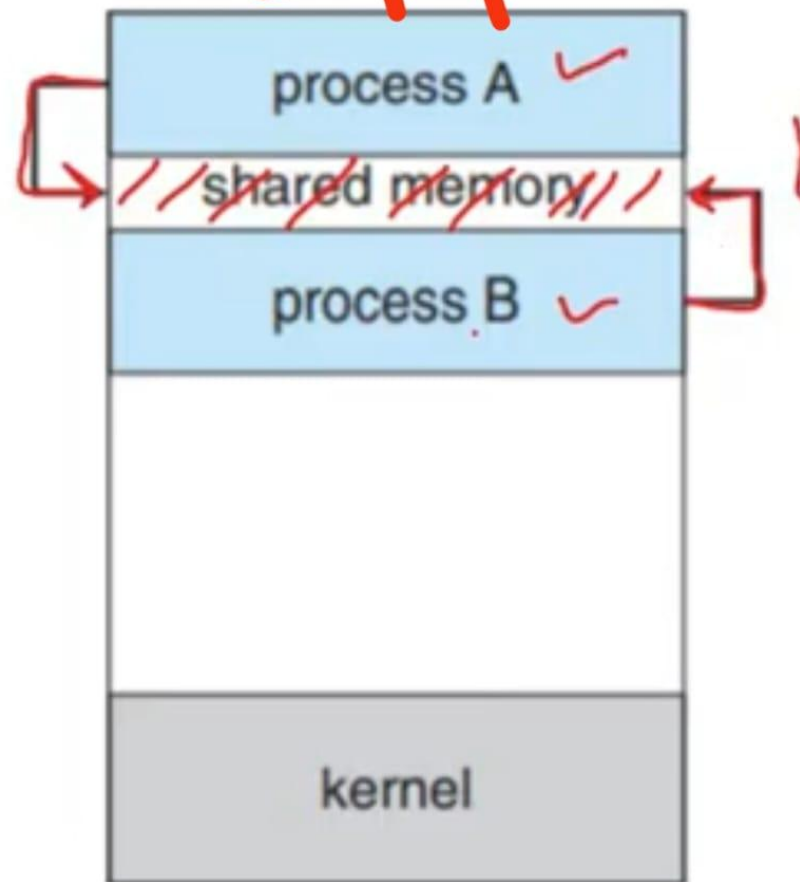
There are two models of interprocess communication: **shared memory** and **message passing**.

Shared Memory: Multiple processes can access a common shared memory. Multiple processes communicate by shared memory, where one process makes changes at a time and then others view the change. Shared memory does not use kernel. **Ex. Producer–Consumer Problem**

m p



SM



(b) Shared memory

Message Passing: Cooperating Processes to communicate with each other via a message-passing facility. Message passing provides a mechanism to allow processes to communicate and to synchronize their actions without sharing the same address space. It is particularly useful in a distributed environment, where the communicating processes may reside on different computers connected by a network.

A message-passing facility provides at least two operations:

✓ `send(message)`

✓ `receive(message)`

- Messages sent by a process can be either fixed or variable size.
- If **fixed-sized** messages can be sent, the system-level implementation is straightforward. This restriction makes the task of programming more difficult.
- **variable-sized** messages require a more complex system-level implementation, but the programming task becomes simpler. This is a common kind of tradeoff seen throughout operating-system

implementation, but the programming task becomes simpler. This is a common kind of tradeoff seen throughout operating-system design.

If processes P and Q want to communicate, they must send messages

 If processes P and Q want to communicate, they must send messages to and receive messages from each other: a communication link (logical) must exist between them. This link can be implemented in a variety of ways. There are several methods for logically implementing a link:

- Direct or indirect communication
- Synchronous or asynchronous communication
- Automatic or explicit buffering

1. Direct Communication:

- `send(P, message)`—Send a message to process P.
`receive(Q, message)`—Receive a message from process Q.

A communication link in this scheme has the following properties:

- A link is established automatically between every pair of processes that want to communicate.
- A link is associated with exactly two processes.
- Between each pair of processes, there exists exactly one link.

Indirect Communication:

The messages are sent to and received from mailboxes, or ports

→ send(A, message)—Send a message to mailbox A.

→ receive(A, message)—Receive a message from mailbox A.

In this scheme, a communication link has the following properties:

- A link is established between a pair of processes only if both members of the pair have a shared mailbox.
- A link may be associated with more than two processes.

2. Synchronous or asynchronous communication:

Message passing may be either blocking or nonblocking —also known as synchronous and asynchronous.

- ✓ • **Blocking send:** The sending process is blocked until the message is received by the receiving process or by the mailbox.
- **Nonblocking send:** The sending process sends the message and resumes operation.
- ✓ • **Blocking receive:** The receiver blocks until a message is available.
- **Nonblocking receive:** The receiver retrieves either a valid message or a null

3. Buffering:

Whether communication is direct or indirect, messages exchanged by communicating processes reside in a temporary queue. Such queues can be implemented in three ways:

- **Zero capacity:** The queue has a maximum length of zero; thus, the link cannot have any messages waiting in it. In this case, the sender must block until the recipient receives the message.
- **Bounded capacity:** The queue has finite length n ; thus, at most n messages can reside in it. If the queue is not full when a new message is sent, the message is placed in the queue and the sender can continue execution without waiting. The link's capacity

is finite, however. If the link is full, the sender must block until space is available in the queue.

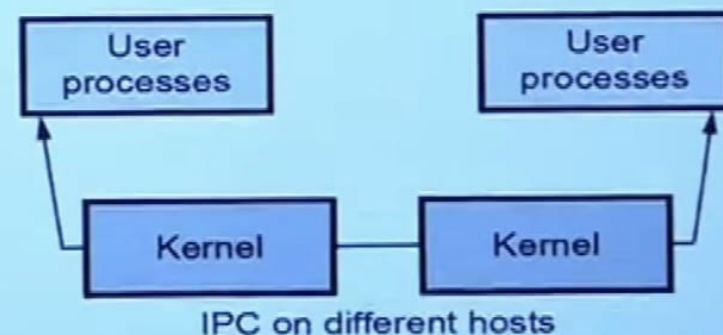
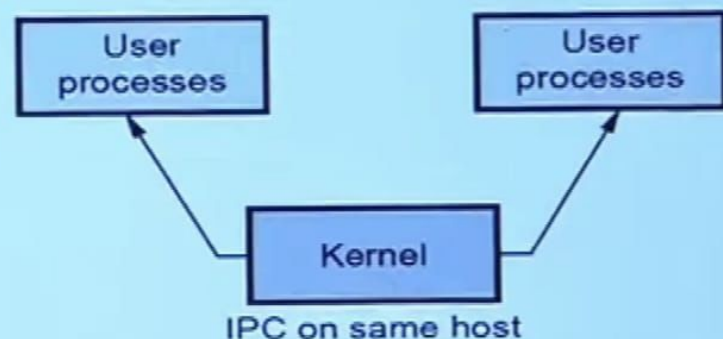
- **Unbounded capacity.** The queue's length is potentially infinite; thus, any number of messages can wait in it. The sender never blocks.

The zero-capacity case is sometimes referred to as a message system with no buffering. The other cases are referred to as systems with automatic buffering.

Inter Process Communication (IPC) using Pipes



$P_1 - P_2$



IPC on different hosts

Types of Pipes

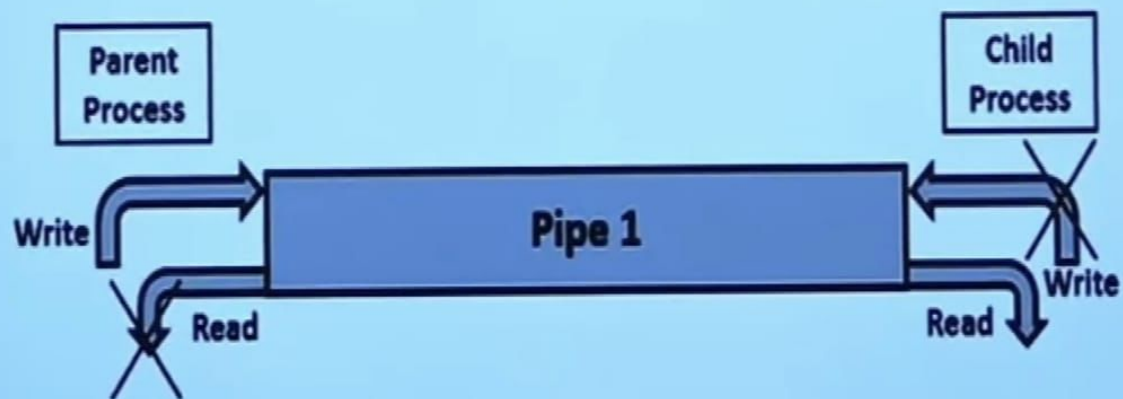
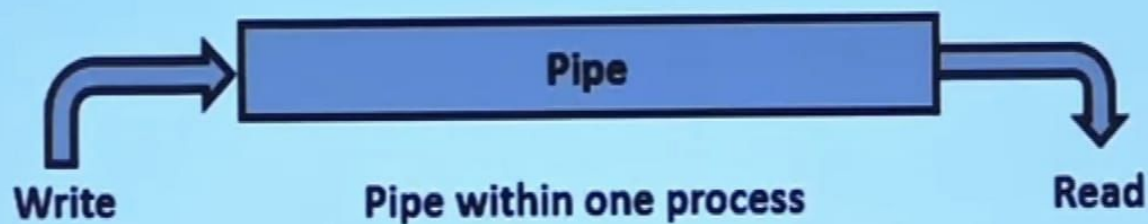
Pipes can be categorized into two types:

unnamed (or anonymous) pipes and

named pipes.

IPC
|
.

Unnamed Pipes



```
#include<stdio.h>
#include<unistd.h>
#include<stdlib.h>
int main()
{
int fd[2],n;
char buffer[100];
pid_t p;
pipe(fd); //creates a unidirectional pipe with
two end fd[0] and fd[1]
p=fork();
if(p>0) //parent
{
Close(fd[0]);
printf("Parent Passing value to child\n");
```

```
write(fd[1],"hello\n",6); //fd[1] is the write
end of the pipe
wait();
}
else // child
{
close(fd[1]);
printf("Child printing received value\n");
n=read(fd[0],buffer,100); //fd[0] is the read
end of the pipe
write(1,buffer,n);
}
}
```

IPC Using FIFO

The entire IPC process will consist of three programs:

Program1: to create a named pipe

Program2: process that will write into the pipe (sender process)

Program3: process that will receive data from pipe (receiver process)

Create a named pipe

```
#include<stdio.h>
```

```
#include<sys/types.h>
```

```
#include<sys/stat.h>
```

```
int main()
```

```
{
```

```
    int res;
```

```
    res = mkfifo("fifo1",0777); //creates a named pipe with the name fifo1
```

```
    printf("named pipe created\n");
```

```
}
```

Sender Process

```
#include<unistd.h>
#include<stdio.h>
#include<fcntl.h>
int main()
{
    int res,n;
    res=open("fifo1",O_WRONLY);
    write(res,"Message",7);
    printf("Sender Process %d sent the data\n",getpid());
}
```

Receiver Process

```
#include<unistd.h>
#include<stdio.h>
#include<fcntl.h>
int main()
{
    int res,n;
    char buffer[100];
    res=open("fifo1",O_RDONLY);
    n=read(res,buffer,100);
    printf("Reader process %d started\n",getpid());
    printf("Data received by receiver %d is: %s\n",getpid(), buffer);
}
```

Advantages & Disadvantages of PIPES

Advantages

Simplicity

Efficiency

Reliability

Flexibility

Disadvantages

Limited capacity

Unidirectional

Synchronization

Limited scalability

RACE Condition

```
int Shared = 5
```

P₁

X = Shared;

X++;

Sleep(1);

Shared = X;

P₂

int Y = Shared;

Y--;

Sleep(1);

Shared = Y;

Process Synchronization:

- Concurrent access to shared data may result in data inconsistency
- Maintaining data consistency requires mechanisms to ensure the orderly execution of cooperating processes.

Race Condition:

- A race condition occurs when two threads access a shared variable at the same time. The first thread reads the variable, and the second thread reads the same value from the variable.
- A race condition is a situation that may occur inside a critical section. This happens when the result of multiple thread execution in critical section differs according to the order in which the threads execute.

Critical Section Problem:

- Consider a system consisting of n processes.
- Each process has a segment of code, called a **critical section**, in which the process may be changing common variables, updating a table, writing a file, and so on.
- The important feature of the system is that, when one process is executing in its critical section, no other process is to be allowed to execute in its **critical section**.

-
- That is, no two processes are executing in their critical sections at the same time (**mutually exclusive**) .
 - The section of code implementing this request is the **entry section**. The critical section may be followed by an **exit section**. The remaining code is the **remainder section**.

Solution of Critical Section Problem:

1. **Mutual Exclusion** - If process P_i is executing in its critical section, then no other processes can be executing in their critical sections. ***No two processes may be simultaneously inside their critical sections.***
2. **Progress** - Progress means that if one process doesn't need to execute into critical section then it should not stop other processes to get into the critical section. ***No process running outside its CS may block other processes.***

$P_2 - 0$
3. **Bounded Waiting** - We should be able to predict the waiting time for every process to get into the critical section. The process must not be endlessly waiting for getting into the critical section. **No process should have to wait forever to enter its CS.**

4. **No assumption related to Hardware** - Our solution must be architectural natural. It means that if our solution is working fine on one architecture then it should also run on the other ones as well. ***No assumptions may be made about speeds or the number***

Of processors

critical section

do {

[Entry section]

critical section

[Exit section]

remainder section

} while(TRUE);

LOCK

critical section solution using lock

do {
 acquire data
 CS
 refuse data
}

execute in user mode
multiprocessor section
No mutual exclusion

ENTRY CODE

1. while (lock == 1);
2. Lock = 1

3. CS

4. Lock = 0

EXIT
CODE

Turn and Set

```
While (lock != 0);
```

```
Lock = 1; ✓
```

Critical Section

```
Lock = 0;
```

```
while (TestAndSet(&lock));
```

Critical Section

```
lock = false;
```



```
boolean TestAndSet (boolean *target)
```

```
{
```

```
    boolean r = *target;
```

```
    *target = true;
```

```
    return r;
```

```
}
```

Turn Variable

Process "P ₀ "	Process "P ₁ "
No CS	
Entry Code → While (turn != 0); Critical Section	While (turn != 1); Critical Section
Exit Code → turn = 1;	turn = 0;

Strict Alteration

Process <u>P₀</u>	Process <u>P₁</u>
<pre>while(1) { while (turn != 0); Critical Section turn = 1; Remainder Section }</pre>	<pre>while(1) { while (turn != 1); Critical Section turn = 0; Remainder Section }</pre>

Peterson's solution

Process P_0	Process P_1
<pre>while (1) { flag[0] = T; turn = 1; while (turn == 1 && flag[1] == T); Critical Section flag[0] = F; Remainder Section }</pre>	<pre>while (1) { flag[1] = T; turn = 0; while (turn == 0 && flag[0] == T); Critical Section flag[1] = F; Remainder Section }</pre>

Producer consumer problem

```
Void Consumer (Void)
```

```
{
```

```
    int itemc;
```

```
    While (true)
```

```
    {
```

```
        While (Count == 0);
```

```
        itemc = Buffer(Out);
```

```
        Out = (Out + 1) mod n;
```

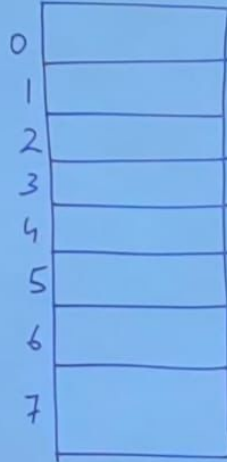
```
        Count = Count - 1;
```

```
        Process_item(itemc);
```

1. Load $R_c, m[Count]$
2. DECR R_c
3. Store $m[Count], R_c$

$n=8$

Buffer[0...n-1]



Count



```
int Count=0;
```

```
Void Producer(Void)
```

```
{
```

```
    int itemp;
```

```
    While (true)
```

```
    {
```

```
        Produce_item(itemp);
```

```
        While (Count == n);
```

```
        Buffer[in] = itemp;
```

```
        in = (in + 1) mod n;
```

```
        Count = Count + 1;
```

- Load $R_p, m[Count]$
- INCR R_p
- Store $m[Count], R_p$

case 1: Normal [without interrupt]

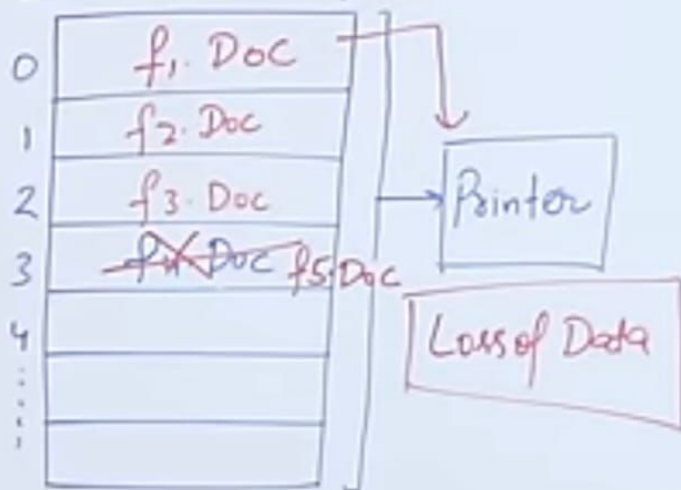
case 2:

producer	consumer	producer	consumer
$I_1 I_2$	$I_1 I_2$	I_3	I_3

Process Synchronization (Printer - Spooler Problem)

- I1. Load $R_i, m[in]$
 I2. Store $SD[R_i], "F-N"$
 I3. INCR R_i
 I4. Store $m[in], R_i$

Spooler Directory



In $\boxed{3} \text{ } 4$

P_1 I₁ I₂ I₃ | P_2 I₁ I₂ I₃ I₄
Preempt

Preempt

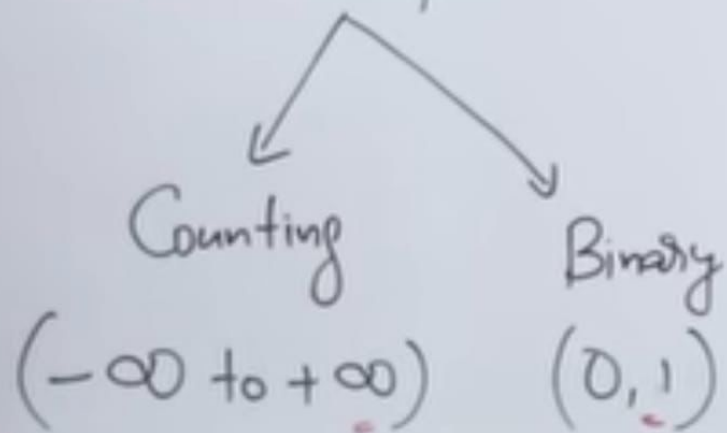
$\boxed{34}$

R_2
 $\boxed{34}$

P_1
 $\boxed{f_4.doc}$

P_2
 $\boxed{f_5.Doc}$

Semaphore



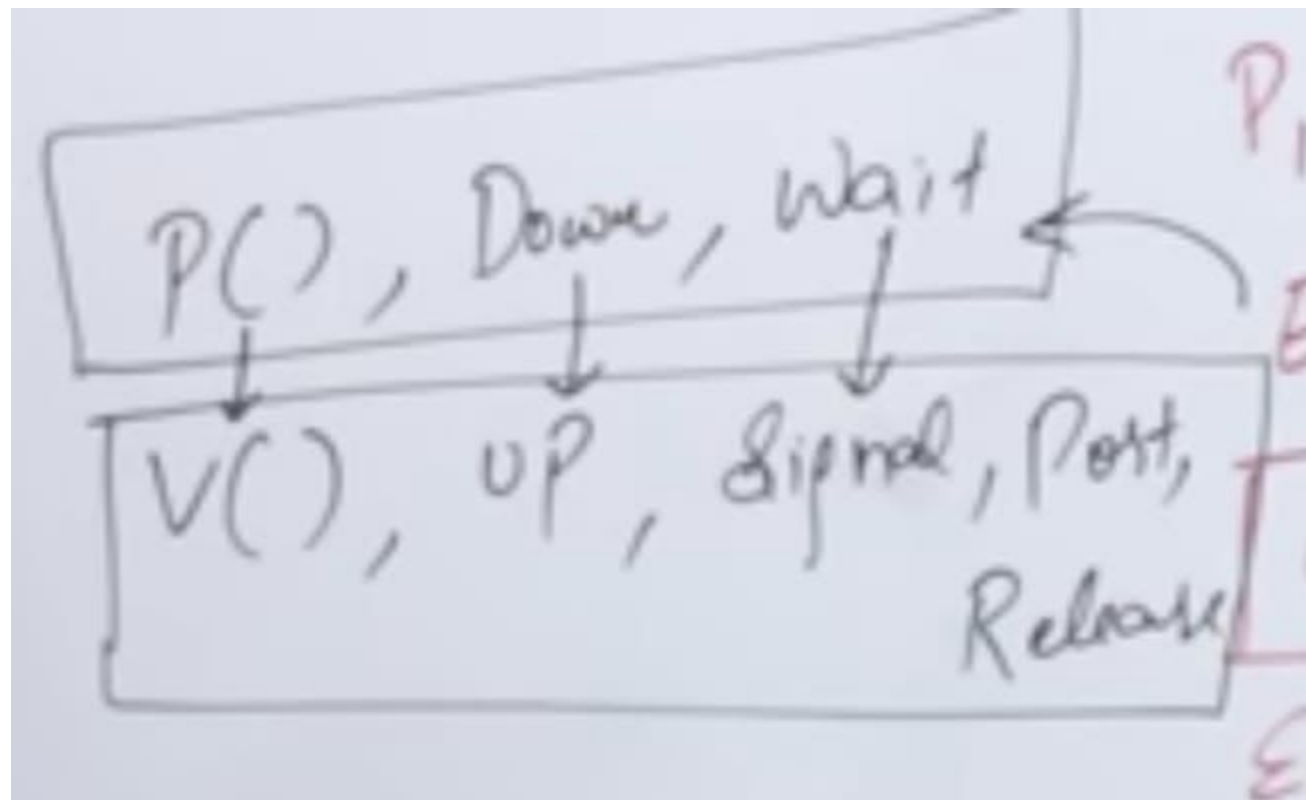
"Semaphore is an integer variable
Which is used in mutual exclusive manner
by various concurrent cooperative processes
in order to achieve synchronization"

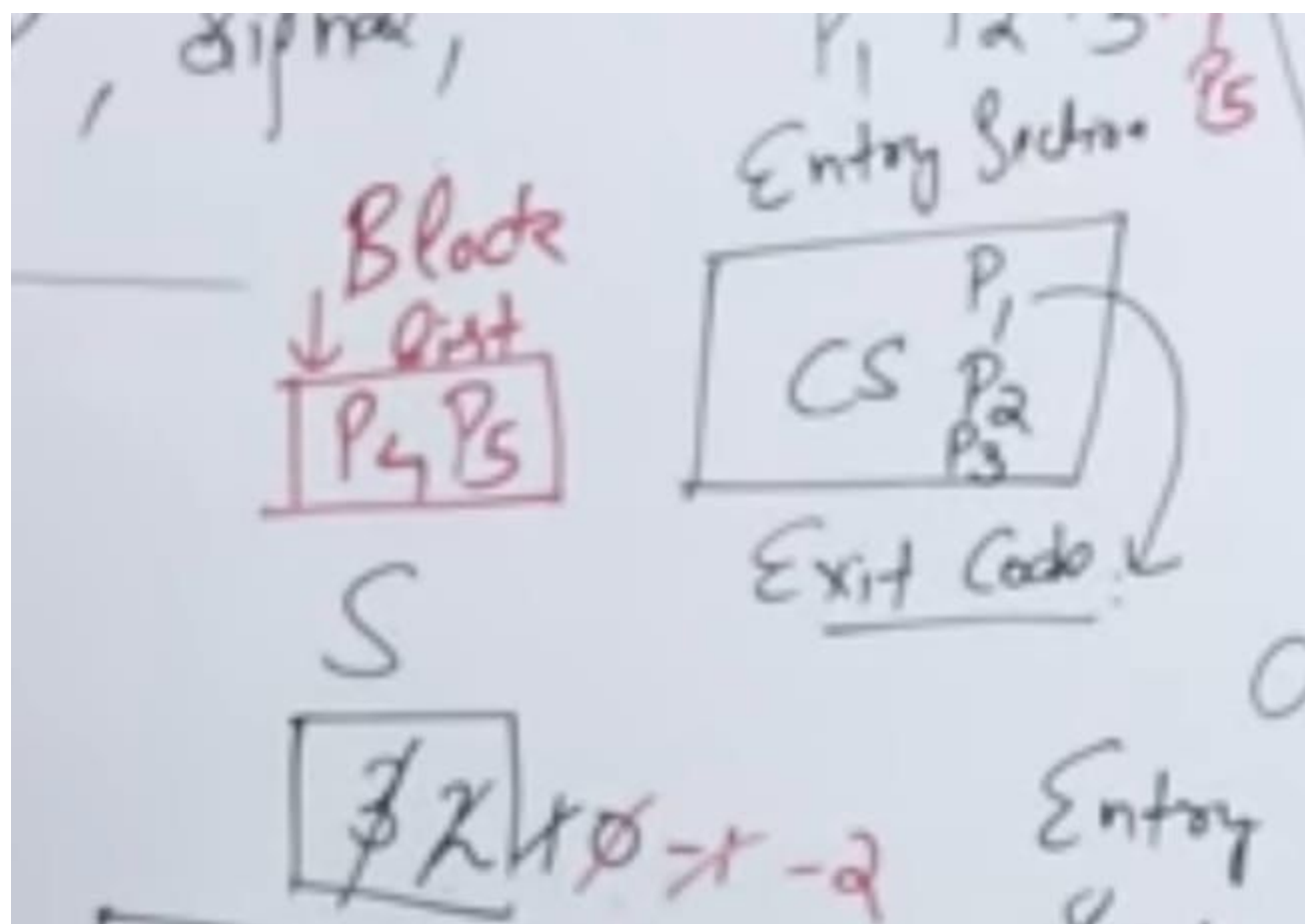
Down(Semaphore S)

```
{
  S.Value = S.Value - 1;
  if (S.Value < 0)
  {
    Put Process (PCB) in
    Suspended list Sleep();
  }
  else
  {
    return;
  }
}
```

UP(Semaphore S)

```
{
  S.Value = S.Value + 1;
  if (S.Value <= 0)
  {
    Select a Process
    from Suspended list
    Wake up();
  }
}
```





S

17

5P, 3V, 1P.

Counting Semaphore $\rightarrow$ full = 0 = No. of filled slots
Binary Semaphore $S=1$; $\rightarrow$ Empty = N $\rightarrow$ No. of Empty slots

Produce_item(item p);

down(Empty);

down(S);

Buffer[IN] = item p;

IN = (IN + 1) mod n;

up(S);

up(full);

IN
3

N = 8

0	a
1	b
2	c
3	
4	
5	
6	
7	

Out
0

Consumer
=

down(full);

down(S);

item C = Buffer[out];

Out = (Out + 1) mod n;

up(S)

up(Empty);

=

int rrc = 0

Semaphore mutex = 1;

Semaphore db = 1;

Void Reader(Void)

{ while (true)

{

down(mutex);

rrc = rrc + 1;

if (rrc == 1) then down(db);

up(mutex)

DB

down(mutex)

rrc = rrc - 1;

if (rrc == 0) then up(db);

up(mutex)

Process_data

{

}

Void Writer(Void)

{

while (true)

{

down(db);

DB

up(db);

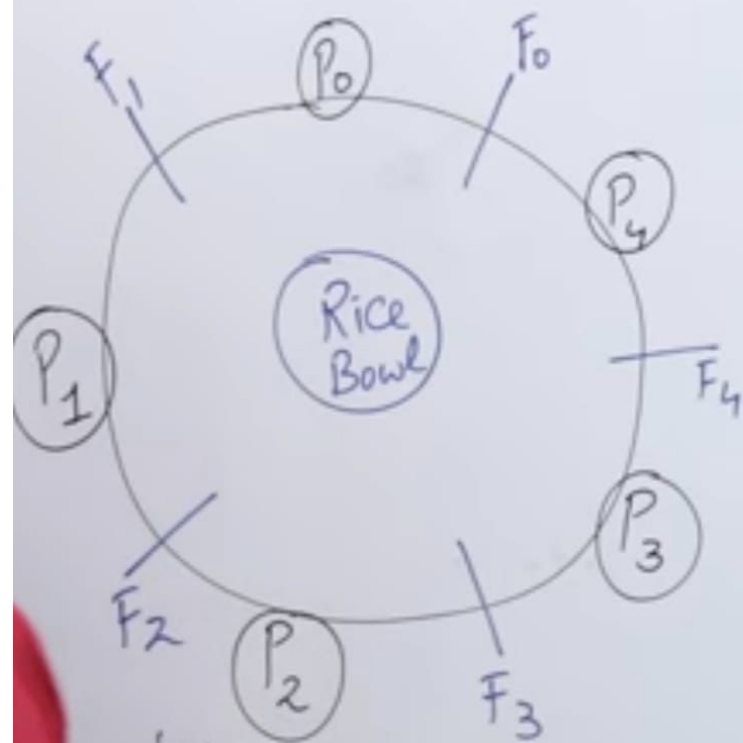
{ }

R - W $\rightarrow$ Problem

W - R - Problem

W - W - Problem

R - R - No Problem



'5' Philosophers ✓
'5' Forks ✓

Void philosopher (void)

```
{
    while (true)
    {
        Thinking();
        take_fork(i);
        take_fork((i+1) % N);
        EAT();
        Put_fork(i);
        Put_fork((i+1) % N);
    }
}
```

Void Philosopher (Void)

```

{
  while (true)
  {
    Thinking();
  }
}

```

P_0 P_1 P_2
 $\downarrow$ $\downarrow$
 Block

Wait (take_fork(S_i))
 Enter
 Wait (take_fork($S_{(i+1) \% N}$))

EAT();

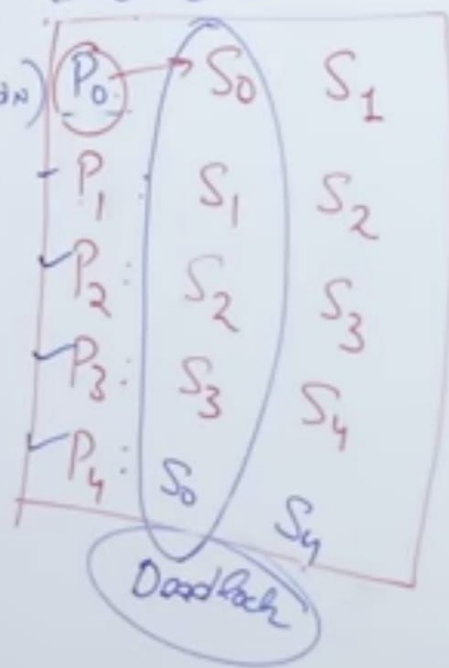
Signal(Put_fork(i));

Signal(Put_fork($(i+1) \% N$));

}

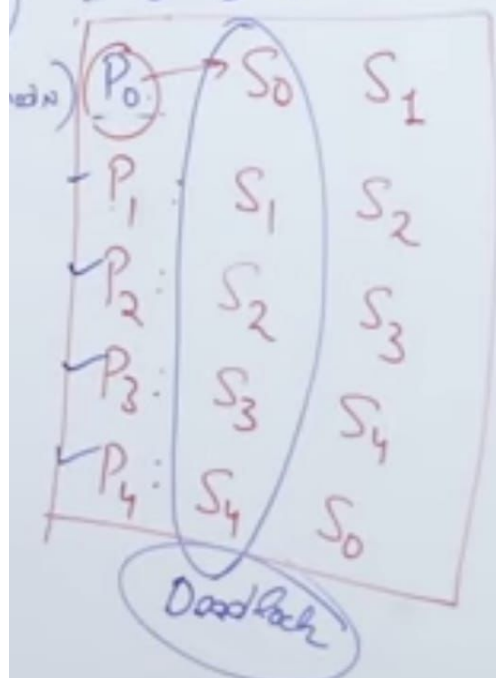
$S[i]$ five Semaphores

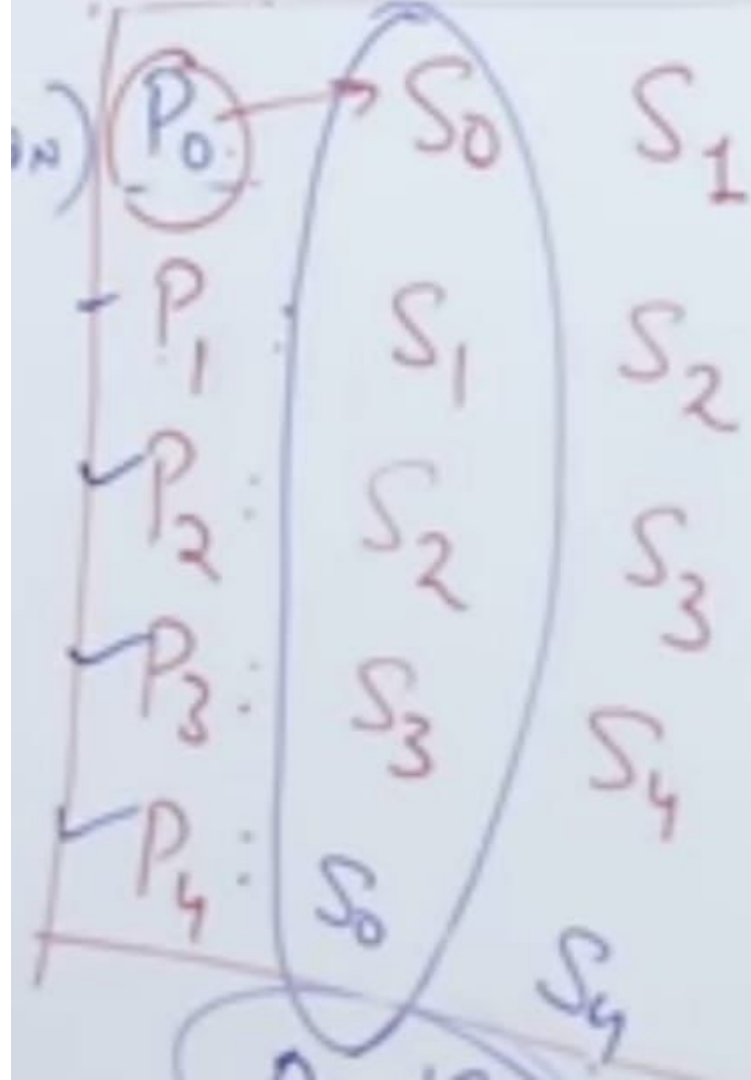
S_0	S_1	S_2	S_3	S_4
$\downarrow$	$\downarrow$	$\downarrow$	$\downarrow$	$\downarrow$
$\times$	$\times$	$\times$	$\times$	$\times$
0	0	0	0	0



$Q[i]$ five semaphore

S_0	S_1	S_2	S_3	S_4
↓	↓	↓	↓	↓
✗	✗	✗	✗	✗
0	0	0	0	0





$$\text{Empty} = \cancel{5} 4$$

$$\text{full} = \cancel{3} 4$$

$$S = \cancel{+} \emptyset 1$$

$$I_n = (3+1) \bmod 8 = 4 \bmod 8 \\ = \textcircled{4}$$

Process P_0	Process P_1
<pre>do { flag[0] = true; while (flag[1]) { if (turn == 1) { flag[0] = false; while (turn == 1); flag[0] = true; } }</pre>	<pre>do { flag[1] = true; while (flag[0]) { if (turn == 0) { flag[1] = false; while (turn == 0); flag[1] = true; } }</pre>

```
}  
    Critical Section  
turn = 1;  
flag[0] = false;  
    Remainder Section  
} while (true);
```

```
}  
    Critical Section  
turn = 0;  
flag[1] = false;  
    Remainder Section  
} while (true);
```

	user mode	s/w mechanism	easy waiting	only two processes	more than two processes	mutual exclusion	progress	Boundedness	possibility
LOCK	✓	✓	✓		✓	X			
Strict Alternation	✓	✓	✓	✓		✓	X	✓	✓
Turnstile	✓	✓		✓		✓	X	✓	✓
Peter's	✓	✓	✓	✓		✓	✓	✓	✓

Monitors

- A high level abstraction that provides a convenient and effective mechanism for process synchronization.
- A monitor type presents a set of programmer-defined operations that provide mutual exclusion within the monitor.
- The monitor type also contains the declaration of variables whose values define the state of an instance of that type, along with the bodies of procedures or functions that operate on those variables.



Syntax of a Monitor

```
monitor monitor_name
{
  // shared variable declarations
  procedure P1 ( ... ) {
    ...
  }
  procedure P2 ( ... ) {
    ...
  }
  .
  .
  procedure Pn ( ... ) {
    ...
  }
  initialization code ( ... ) {
    ...
  }
}
```

- A procedure defined within a monitor can access only those variables declared locally within the monitor and its formal parameters.
- Similarly, the local variables of a monitor can be accessed by only the local procedures.
- The monitor construct ensures that only one process at a time can be active within the monitor.

Condition Construct-

condition x, y;

The only operations that can be invoked on a condition variable are **wait ()** and **signal ()**.

The operation **x.wait()**; means that the process invoking this operation is suspended until another process invokes **x.signal ()**;

The **x. signal ()** operation resumes exactly one suspended process.

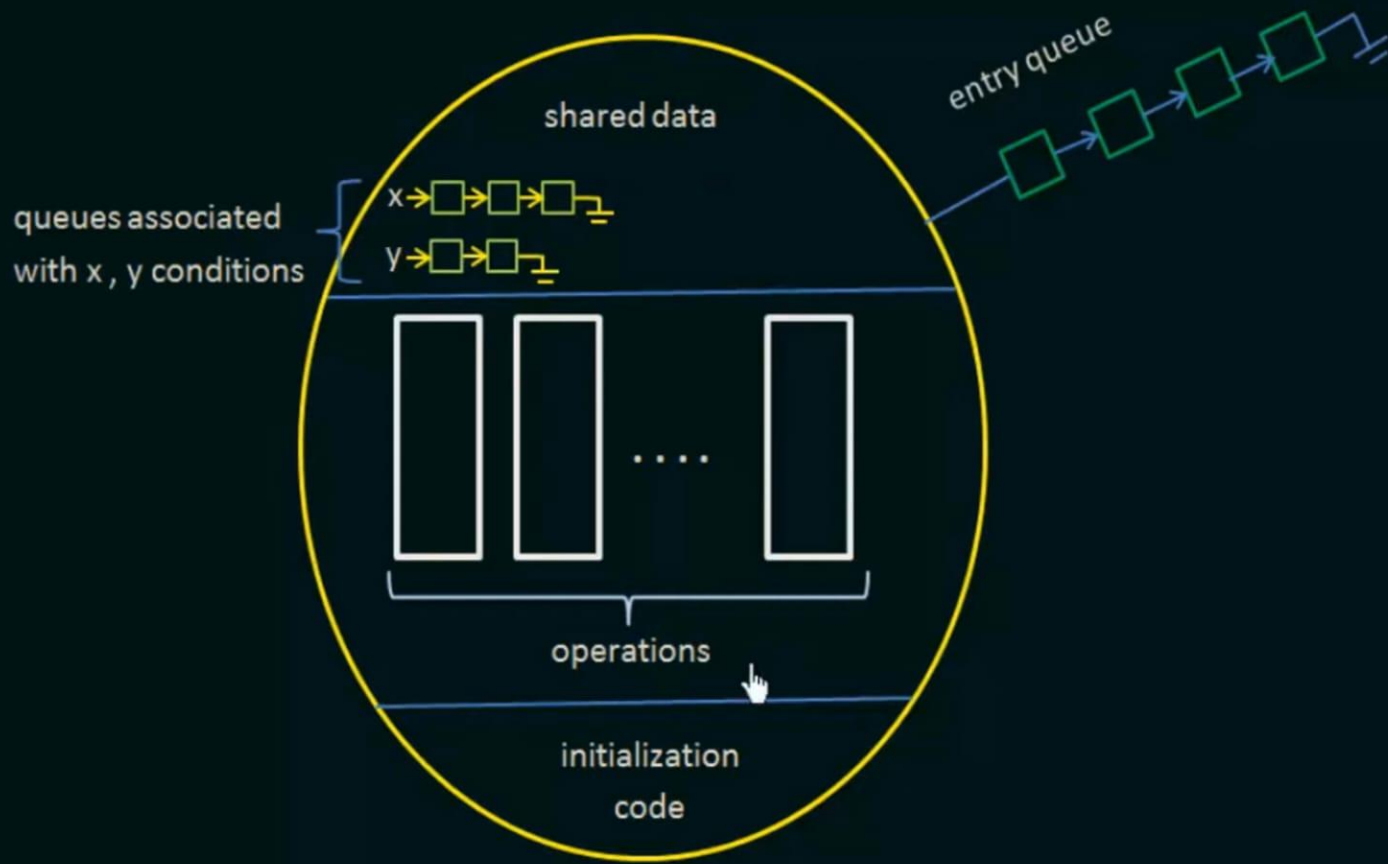


Fig: Schematic view of a monitor

Dining-Philosophers Solution Using Monitors

- We now illustrate monitor concepts by presenting a deadlock-free solution to the dining-philosophers problem.
- This solution imposes the restriction that a philosopher may pick up his chopsticks only if both of them are available.
- To code this solution, we need to distinguish among three states in which we may find a philosopher. For this purpose, we introduce the following data structure:

```
enum { thinking, hungry, eating } state [5] ;
```

- Philosopher i can set the variable $state[i] = eating$ only if his two neighbors are not eating: $(state[(i+4) \% 5] \neq eating)$ and $(state[(i+1)\% 5] \neq eating)$.
- We also need to declare `condition self [5] ;` where philosopher i can delay himself when he is hungry but is unable to obtain the chopsticks he needs. 

```
monitor dp {  
    enum { THINKING, HUNGRY, EATING } state [5];  
    condition self [5];  
  
    void pickup (int i) {  
        state [i] = HUNGRY;  
        test (i);  
        if (state [i] != EATING)  
            self [i].wait();  
    }  
  
    void putdown(int i) {  
        state [i] = THINKING;  
        test ((i + 4) % 5);  
        test ((i + 1) % 5);  
    }  
  
    void test (int i) {  
        if ((state [(i + 4) % 5] != EATING) &&  
            (state [i] == HUNGRY) &&  
            (state [(i + 1) % 5] != EATING)) {  
            state [i] = EATING;  
            self [i].signal();  
        }  
    }  
  
    initialization-code () {  
        for (int i = 0; i < 5; i++)  
            state [i] = THINKING;  
    }  
}
```